NEA - Group Creator for School Projects

Amelie Machin

Contents

1 Analysis **2**
1.1 Problem statement . . . 2
1.2 Research . . . 2
1.2.1 Sources of inspiration . . . 2
1.2.2 Algorithm . . . 3
1.2.3 Inputting data . . . 4
1.2.4 Accounts . . . 4
1.2.5 Internal data storage . . . 5
1.2.6 External data storage . . . 5
1.2.7 GUI library . . . 5
1.3 Dialogue . . . 6
1.3.1 Initial interview . . . 6
1.3.2 Follow-up questions . . . 7
1.4 Objectives . . . 8
1.5 Modelling . . . 14

2 Design **17**
2.1 GUI . . . 17
2.2 Databases . . . 24
2.3 Inputs and outputs . . . 24
2.4 Algorithm . . . 26
2.4.1 Pseudocode . . . 26
2.4.2 Generating Initial Groups . . . 28
2.4.3 Fitness algorithm . . . 30
2.4.4 Diagrams . . . 30

3 Technical solution **32**
3.1 main.py . . . 33
3.2 algoClasses.py . . . 53
3.3 geneticAlgo.py . . . 55
3.4 CreateTables.sql . . . 59

4 Testing **60**
4.1 Video . . . 64
4.2 Testing plan . . . 64

5 Evaluation **66**
5.1 Self reflection 66
5.2 Third-party feedback 66

1 Analysis

1.1 Problem statement

To form the perfect grouping for a group project within a school, many factors need to be taken into consideration to prevent students from feeling incapable of completing the project. These factors include, but are not limited to, skills of the students, skills required by the project, and number of students in the group.

This project aims to create a tool to aid teachers in ensuring the students are separated into groups where the students are best able to get along and combine their individual skills to achieve their project's aims. The user can input the data that they want to be taken into consideration by the program and it will output a set of groups. This is in the form of a table on a GUI - to present the results in a readable way - and an option to export the data.

This program will not find, assume, or generate information about the students or project, it will only work with the data inputted to assign a group of students to a project.

Teachers will benefit from this tool as they don't have to spend as much time or effort creating groups for the projects they set for their students, so they can spend their time and effort on helping and teaching the students instead. Also, when the students are working on their projects, they are more likely to be focused on their work, so require less oversight from the teacher.

Students will also benefit from this tool as they would be able to apply their skills to a project in a group of people they can enjoy time with, rather than being stuck on a project they have no interest in with people they do not get along with. This means they are more likely to enjoy their time at school, be able to get more work done, and therefore be able to learn more, as opposed to being unmotivated and distracted in a poorly made group.

1.2 Research

1.2.1 Sources of inspiration

1. Random group generator from ClassTools.net [Fig. 1]
   https://www.classtools.net/random-group-generator/

   - This tool is used frequently by my teachers to create random groups for group activities.
   - The GUI is very simple to use as there is little data that needs to be entered, so can all fit down the left of the screen.

- The output is very easy to read as each group is visually separated using both coloured boxes and numbering. [Obj. 3b]
- However, this tool is too simple because it lacks any way to group with conditions, which is part of my scope, as they are purely randomised. This means that teams could quit often be unfairly balanced in terms of skill, or group students together that are known to not work well together. The teacher's only option to solve this is to re-roll until the output is suitable enough for the task manually. This gave me the idea for the algorithm to be the simulation of this re-rolling and selection of optimal outputs.

2. keamk [Fig. 2] [Fig. 3]
   https://www.keamk.com/random-team-generator
   - This tool creates groups that all have a balanced level of skill.
   - I like how the skill level is inputted as a rating between 1-5 as this is intuitive for the user.
   - I also like the extra details that are outputted, such as how many are in each group next to the group name and the skill levels next to each individual. [Obj. 3a]
   - Furthermore, the option to go back to make small changes to the data inputted and get new groupings was very quick and useful. [Obj. 1g]
   - However, this tool can only create balanced groups, not groups with different objectives as is in my scope, and there is no way to assign a certain number of participants to each group, as the program puts the same number (or one away) of participants in each group. [Obj. 2c-d]

These tools influenced me to reduce the complexity of my GUI, and gave me ideas such as to keep groups visually separate using colour as well as tables, adding the option to go back and edit the inputted data later, and some summary statistics to include with the outputs.

1.2.2 Algorithm

I have researched into representing students and projects into bipartite graphs, and then using matching algorithms such as the alternate path algorithm to join students to projects [Fig. 4]. However, this will only match one student to one project, not groups so is not suitable. I then researched applying the algorithm repeatedly to graphs with randomised students, but this means that some students will never be joined into a group purely based on chance and that the groups formed are such that each individual is best for the project, not that the overall group is best suited for the project, so teamwork will not as good as it could be. I then researched using a grouping algorithm, such as https://help.feedbackfruits.com/hc/en-us/articles/23527084211346-Group-

Formation-Algorithm-Explainer to create similar groups then using bipartite matching algorithms to match them. However this only creates groups with similar skill sets, not skill sets that vary based on the project.

I decided on a genetic algorithm where each individual is made up of a possible solution and the fitness score is an average between how suited each group is to its assigned project [Fig. 5]. The individuals with higher fitness scores are discarded and the lower ones are randomly selected to crossover to form new individuals. Some of these individuals formed may contain error such as the same student multiple times, or a missing student, so that must be corrected before continuing. Then, random mutations are applied to some individuals, which may involve swapping of students between groups, moving some students to an optional project, or moving some students to group that isn't full. New fitness scores are calculated and the process repeats a predetermined number of times [Fig. 9]. I chose this because genetic algorithms are very adaptable to varying conditions and constraints, such as the ones that the user chooses in this program. This will allow the program to test different combinations of projects and students, test different group sizes for a project, and swap in and out optional projects to find better and better solutions until it reaches a high-quality solution that fits all the user's needs. This fits the project scope far better than all previously researched algorithms.

1.2.3 Inputting data

For the inputting of data, I have decided that it can be inputted through txt and csv files [Obj. 1a], manually on the GUI [Obj. 1b], or any combination of those methods [Obj. 1c]. This is because the data for different individuals and different projects could be stored in different ways within the same system, and manual tweaks may be needed by the user to ensure all data is as is needed by the application. I will also include the option to import the data from an Excel spreadsheet at the request of my third party [Obj. 1b].

1.2.4 Accounts

I have decided against implementing an account system because I don't think it will provide any value to the program. Accounts are typically used for ensuring data can only be accessed by the users with the correct permissions. As this program will be designed to be run locally, all the data will be stored and only accessible on the machine the program was run on, so the OS can handle user security.

Another reason for accounts is allow users to access their data that has been stored by a third party, especially within a cloud to allow for data to be quickly transferred between devices. I will implement a manual solution to this by allowing users to export and import all the data the program stores and the user can handle that data transfer themselves.

1.2.5 Internal data storage

I chose to store the data about the students and projects in their own classes, that are aggregated within an 'individual' class. The student and project classes contain their names, skills and the project will have the recommended group size. The individual will contain a project object, student objects, and current fitness score as well as methods for calculating fitness scores, crossover with other individuals, and mutations. [Fig. 7]

This keeps the attributes and methods stored together in the program to make the code easier to read and understand by a programmer.

1.2.6 External data storage

[Obj. 4] I have researched into whether information about projects and students should be stored somewhere, and if so, how. I have concluded that they should be stored in relational databases that are accessible and modifiable by entering the details into the graphical interface, which are passed as parameters into SQL statements. These can then be loaded as a list of checkboxes that can be selected and deselected by the user for that run. [Obj. 1c]

I chose to store the data because being able to make minor changes to the data and then re-running the program quickly is a very valuable feature that could save users' time and energy as they would not have to go through the entire process of entering every piece of data just for small changes to some of the data. This feature is present in keamk's grouping tool, and I found it very useful in my testing.

I chose databases to store the data because I have more experience working with relational databases and SQL then any other form of data storage, such as in files. The data is structured so databases would allow for efficient access and modification of specific entries.

1.2.7 GUI library

Between the two main python GUI libraries, pygame and Tkinter, I chose Tkinter because of the larger library of classes of pre-made, customisable widgets that can be used more easily to produce a simple, easy-to-use tool, such as a dialog that allows users to graphically navigate through their files to find their data file [Fig.6].

On the other hand, with my prior experience with pygame, I have found it to be more suited towards building games and graphically-complex tools, which is not what I am aiming for in this project. I am aiming to have a GUI that is quick and simple to create and edit, and has user-friendly features for inputting, editing, and reading data. I also have more experience with Tkinter, which will reduce the amount of time I spend on the GUI and allow me to focus on developing and improving the grouping algorithm and user experience.

1.3 Dialogue

My third party is Dr Ed Horncastle, the lead teacher for the Exeter Mathematics Certificate (EMC) at my school, which is where groups of 3 or 4 students work together on research projects set by University academics or companies. He is suitable because he has to make these groups based on the skills of the students and match them to projects every year, so he has a thorough understanding of how this tool could work and would benefit from using this tool.

1.3.1 Initial interview

The following questions were asked before I started to design the program:

1. What is the current process for creating EMC groups?
    - All the metrics to consider are gathered into a spreadsheet and then groups are formed by hand.
    - Some projects don't end up getting groups assigned, especially those that have been set by a company that already has another project with an assigned group.
2. What are these metrics?
    - Students' own choice
    - There must be a mix of genders
    - The projects must be relevant to the A-level studies of that student. Someone who does not take computer science shouldn't be given a computer science based project
    - Those that work hard and those that laze off more easily should be mixed together. If this is their first project, this is inferred from GCSE results, otherwise it is based off the results of previous projects.
    - Special educational needs
    - Timetabling restrictions
    - Students do not get put into groups with others they have worked on projects with before
3. How are the student choices measured?
    - For some of the projects, the students give their top 3 choices of projects. Starting with the least popular project, the groups are formed by trying to minimise the sum of the positions the student's ranked the project in.
4. How is the data about the students and projects currently stored?
    - Excel spreadsheet
5. How would you prefer to input the data?

- Student choices from a Microsoft Form and all other data from a Excel spreadsheet

6. How much data and in what format do you think the data should be outputted?
    - In tables with the project name, student names and summaries of information about each group such as gender ratio, how many take computer science or physics, and how many have special education needs.
    - As a simplified register in alphabetical order
7. How useful would storing project/student details locally, that can be loaded when needed, be for you?
    - Very
8. Are there any other features that would be useful for this tool to include?
    - Some extra features that could be added is timetabling for what order the projects are presented in, and itinerary for group trips out of school.

After this, I decided to include Excel spreadsheets as an input method [Obj. 1b], include options for projects to not have groups assigned despite being a part of the algorithm [Obj. 1f], take genders into account [Obj. 2e], have summary statistics presented alongside the groups [Obj. 3a], have the option to view the groups without any extra information [Obj. 3b], store the data locally for re-use at another time [Obj. 4].

I decided not to include student choice, student personality, special education needs, timetabling restrictions, or preventing grouping the same students together across multiple as adding too many restrictions to the algorithm will make it take it longer to run to create a set of groups that best matches all of these restrictions. These are the restrictions that were lower on Dr Horncastle's priorities. I will also not include timetabling or scheduling as these deviate quite far from my project's scope.

1.3.2 Follow-up questions

The following questions were asked during designing or coding the technical solution:

1. I have decided on using three factors to determine how "good" a group is: the number of students with at least one relevant skill to the tasked project; the number of skills required by the project that the group meets; whether or not there are at least two different genders in the group. Could you rate how important these factors are in your opinion, or whether you think any are equal?
    -

1.4 Objectives

Objectives beginning with a * are extra objectives, only implemented if time allowed and are not required for the program to meet the scope of the problem.

1. Data can be entered into the program
    (a) Data can be entered manually into the GUI
    (b) Student and project data can be entered through file import
        i. File type can be csv
        ii. *File type can be txt
        iii. *File type can be a SQLite3 database
        iv. *File type can be xlsx
    (c) Data can be loaded from stores from previous uses of the program
    (d) Data can be entered through a combination of other methods
    (e) Student data must contain a name, gender, and skills
        i. Name must only contain alphanumeric characters and certain whitelisted characters
    (f) Project data must contain a name, maximum group size, minimum group size, whether or not the project must have a group or not, and skill requirements
        i. Name must only contain alphanumeric characters and certain whitelisted characters
        ii. Maximum group size must be greater than 0 and greater than minimum group size
        iii. Minimum group size must be greater than 0
    (g) Settings data must contain a target fitness, max number of iterations, and the size of the populations
        i. Target fitness but be between 0 and 1
        ii. Max iterations must be a positive integer, not including 0
        iii. Size of populations must be a positive integer, not including 0
    (h) Project and student data can be added, edited, and removed at any time before the genetic algorithm runs
    (i) At least one student, at least one project, target fitness, max iterations, and size of populations must all be entered/have values before the algorithm is run
2. Students must be grouped together and to a project
    (a) All students must be put into one group each
    (b) *All students must have at least one skill that will allow them to contribute to the project, when possible

(c) Projects must not have more students assigned to it than the maximum limit inputted by the user

(d) Projects must not have less students assigned to it than the minimum limit inputted by the user

(e) *Groups must have a mix of genders, when possible

3. The resulting groups must be viewable

(a) The user must have to option of coloured tables displayed on the GUI

i. These tables contain the project name, student names, number of students, gender ratio, and skill range

ii. Adjacent tables are coloured differently from each other

(b) The user must have the option of a register layout displayed on the GUI

i. Each project and its details are stored in separate boxes with a background distinct from its adjacent neighbours

ii. Each box contains the project name, and the names of students in that group

iii. Projects are displayed in alphabetical order

iv. The students in each group are displayed in alphabetical order

(c) Both above options can be exported to locally stored files

i. File type can be csv

ii. *File type can be txt

iii. *File type can be xlsx

4. All data must be stored

(a) The data about projects, students, and results must be stored in relational databases

(b) The user must have the option to export the databases' contents to locally stored files

i. File type can be csv

ii. *File type can be txt

iii. *File type can be xlsx

(c) The settings data must be stored as a JSON file

5. The design must be consistent across all pages

(a) Text is black with the Arial font

(b) The top of each page that displays data must feature a header

i. The header contains text that provides a title for the data shown

ii. The title text must be displayed on the left with a significantly larger font to the data shown

iii. The header colour must be of hex code #FFDB32

iv. The header may contain buttons, solely for navigation, on the right

(c) Data about different entities must be visually separate by keeping all the data about each single entity within individual boxes

(d) Buttons that perform a function that cannot be performed at that time must be greyed out, and feature text below stating how to enable the button

(e) Buttons that lead to a different page must feature the name of the page, or the relationship between the current page and next (e.g. "back")

(f) Up to 6 colours can be used to visually separate entities through colour. Red, orange, yellow, green, blue, purple. These colours must have an alpha value of between 10% and 50% to ensure the text remains legible

6. Users must be happy with the program

(a) 100% of users must be happy with navigating the program

(b) 100% of users must be happy with response times

(c) 100% of users must consider the groups outputted by the program to be suitable based on the provided data

(d) 100% of users must consider the methods of data entry to be convenient

(e) 100% of users must consider the methods of exporting data to be simple

(f) 100% of users must be able to read all data shown on the GUI

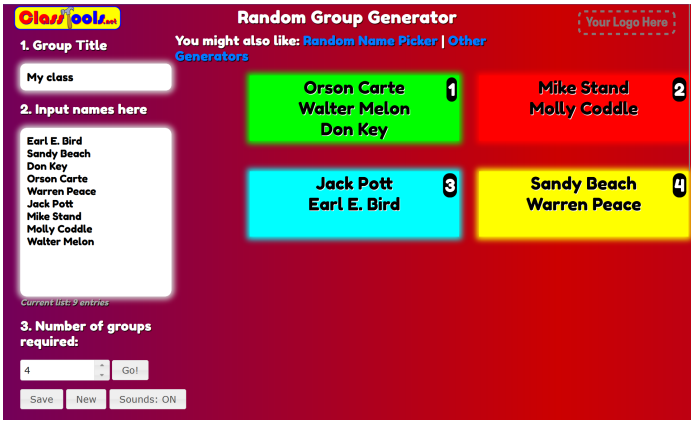


Figure 1: ClassTools's random group generator tool with inputs and outputs visible on the same page

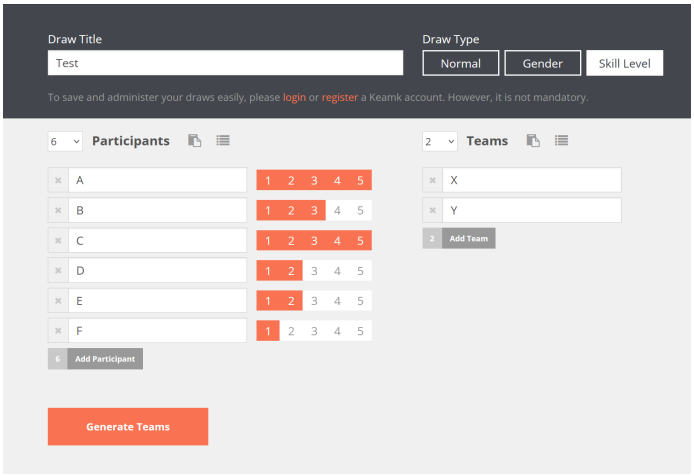


Figure 2: Keamk's menu for inputting data



Figure 3: Keamk's outputs

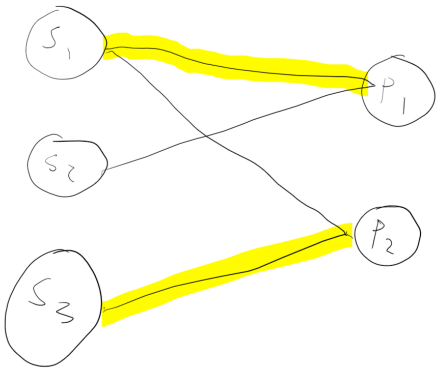


Figure 4: A bipartite graph with 3 student nodes on the left, 2 project nodes and the right, and some edges between them. A matching algorithm has been applied, with the matching highlighted in yellow. This has not met the scope because a student has not been matched.

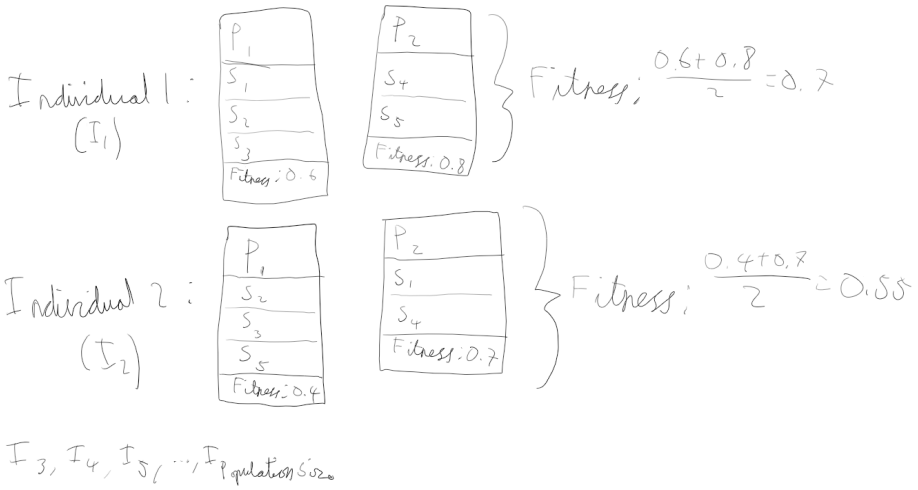


Figure 5: A sketch showing an example of a population, representing a set of possible solutions (individuals). Each individual represents a possible set of groups, and has a fitness score that represents how well every project's group matches the project's and user's requirements. This is calculated by averaging all the fitness scores of the groups it contains.

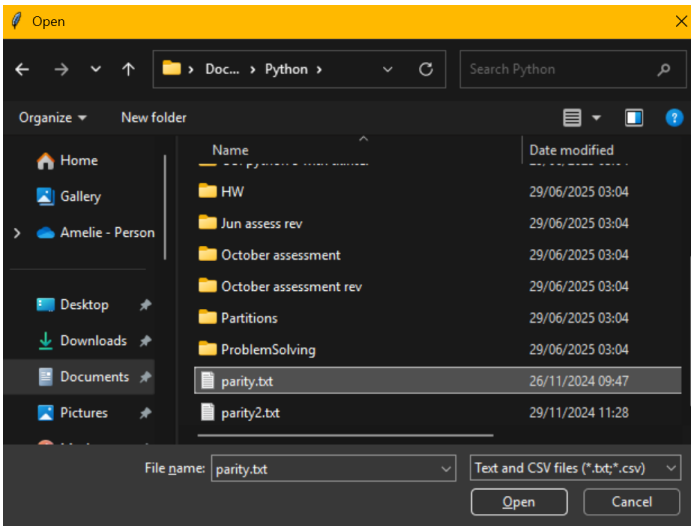


Figure 6: Tkinter's file dialog, configured to only show .txt and .csv files

1.5 Modelling

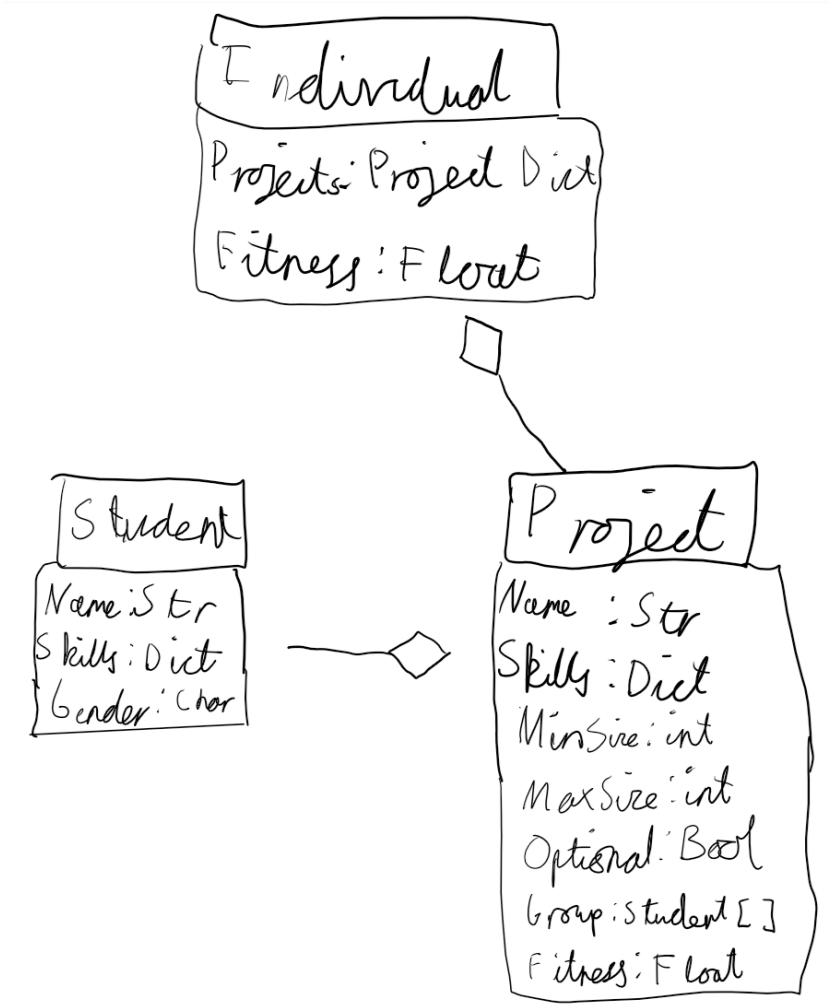


Figure 7: Class diagram showing relationships between the classes and their attributes used by the genetic algorithm.

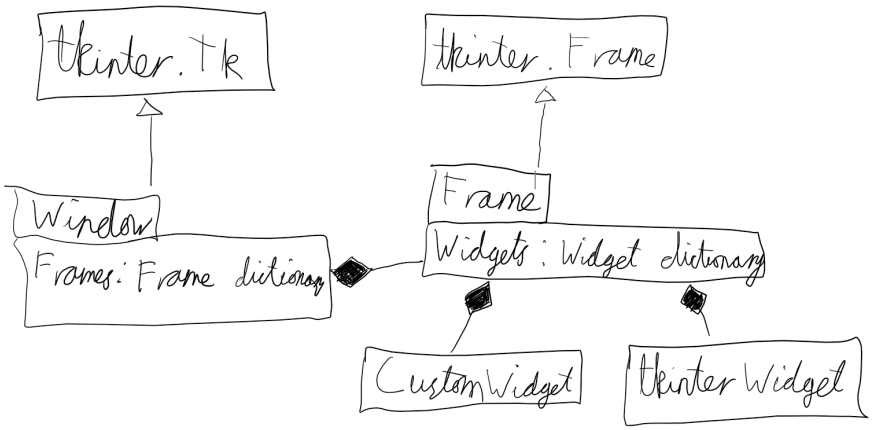


Figure 8: Class diagram showing relationships between the classes that form the GUI

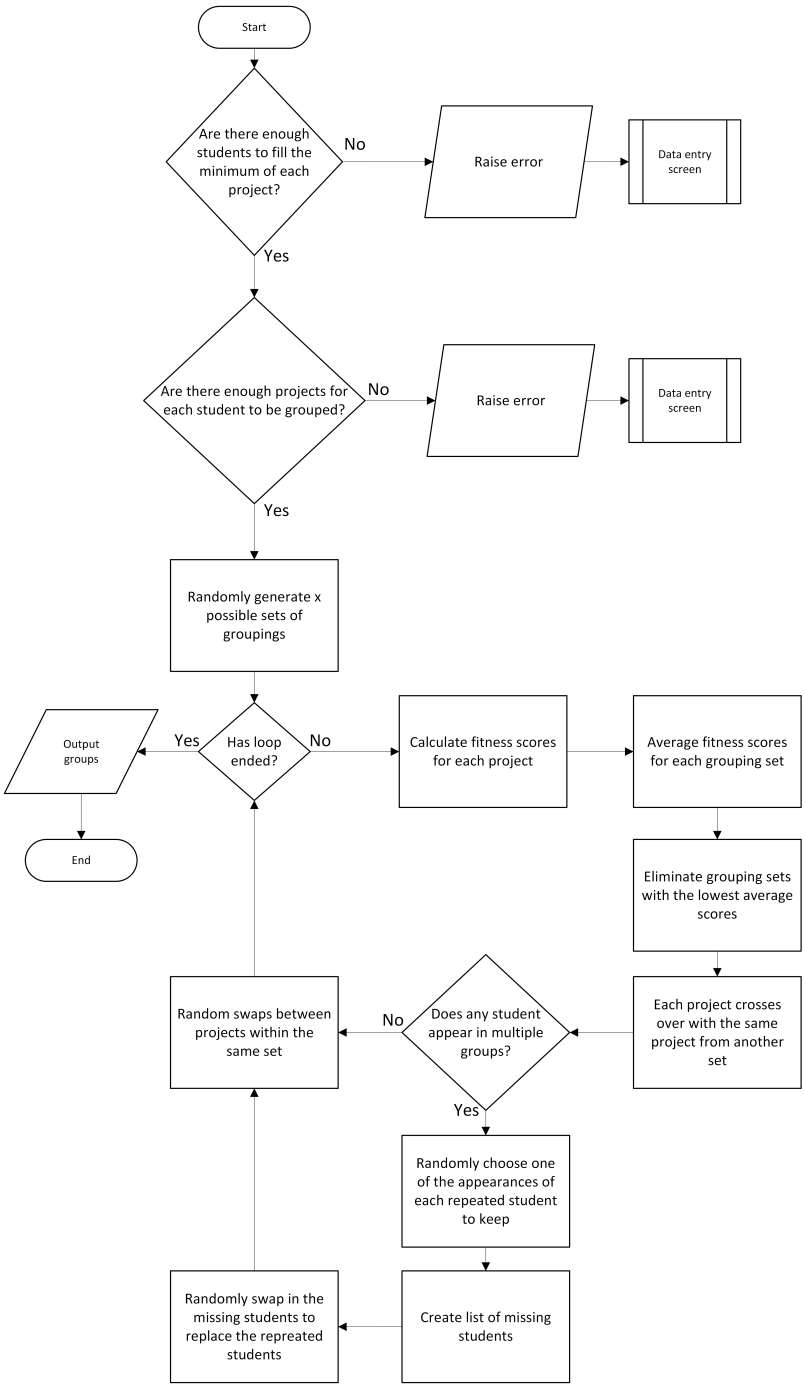


Figure 9: Flowchart showing the process for the genetic algorithm

2 Design

2.1 GUI

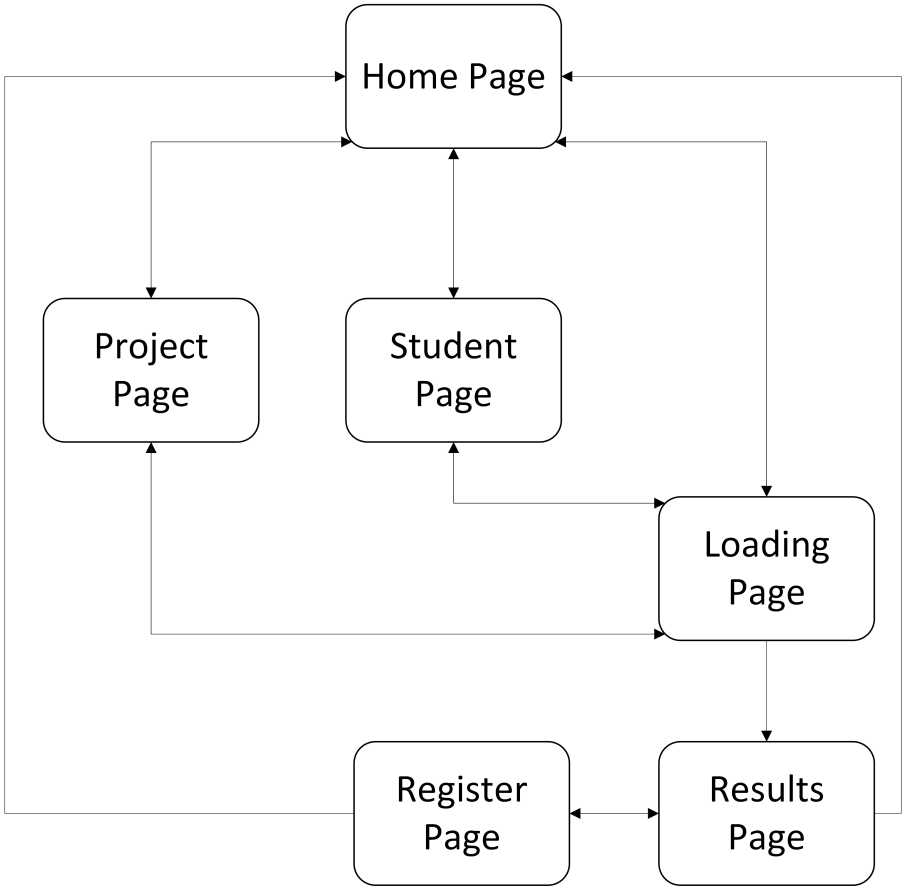


Figure 10: Page navigation diagram showing which pages can navigate to each other. Note: The student and project pages can only navigate to the previous page.

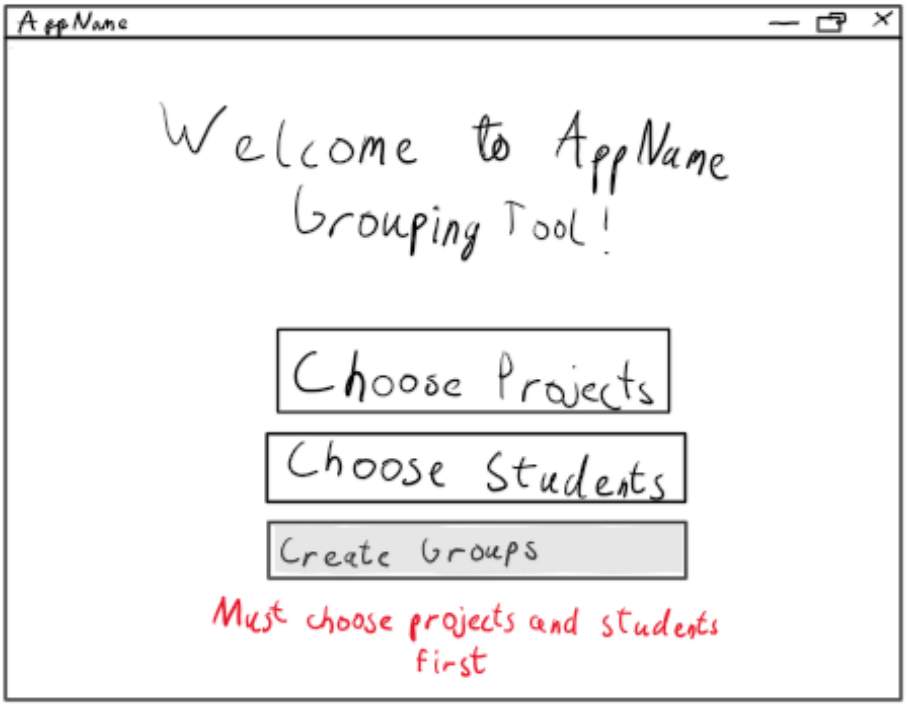


Figure 11: Sketch of the homepage layout that is the first page seen upon opening the app. The "Choose Projects" and "Choose Students" lead to the projects and students pages, respectively. The "Create Groups" button leads to the loading page and runs the grouping algorithm with the students and projects the user has selected. It is greyed out and not clickable unless at least one project and student has been selected [Obj. 1h].

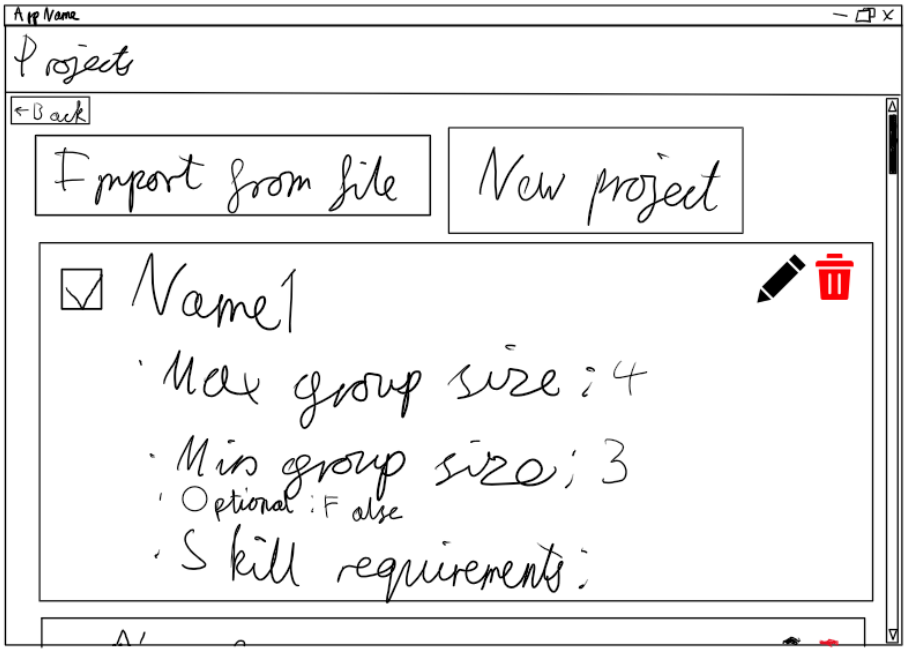


Figure 12: Sketch of the project page layout. The projects in the scrollable list are all those that are currently stored in the database [Obj. 1c]. There are options to import project data from supported files [Obj. 1b] and to manually enter project details at the top [Obj. 1a], which then get added to the database [Obj. 4] and the visual list [Obj. 1d]. The project details can also be edited or deleted from the database by pressing the buttons [Obj. 1g], which run parametrised SQL. Each project in the list features a checkbox, allowing users to select which of the projects in the database will be used in the grouping algorithm. Projects not selected will not have groups matched to them. This page also features a back button to go back to the home page.

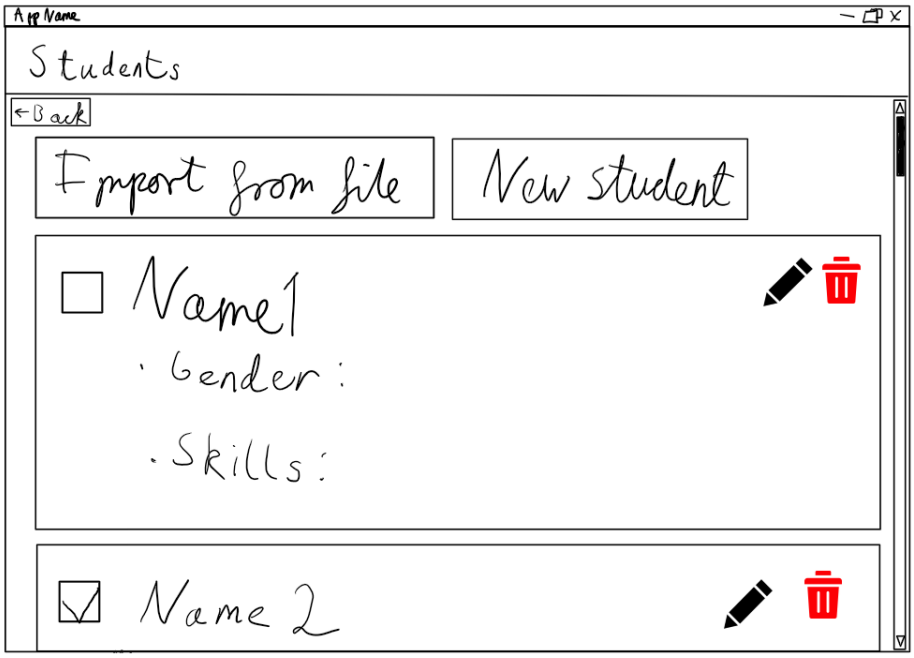


Figure 13: Sketch of the student page layout. The students in the scrollable list are all those that are currently stored in the database [Obj. 1c]. There are options to import student data from supported files [Obj. 1b] and to manually enter student details at the top [Obj. 1a], which then get added to the database [Obj. 4] and the visual list [Obj. 1d]. The student details can also be edited or deleted from the database by pressing the buttons [Obj. 1g], which run parametrised SQL. Each student in the list features a checkbox, allowing users to select which of the students in the database will be used in the grouping algorithm. Students not selected will not be put into groups. This page also features a back button to go back to the home page.

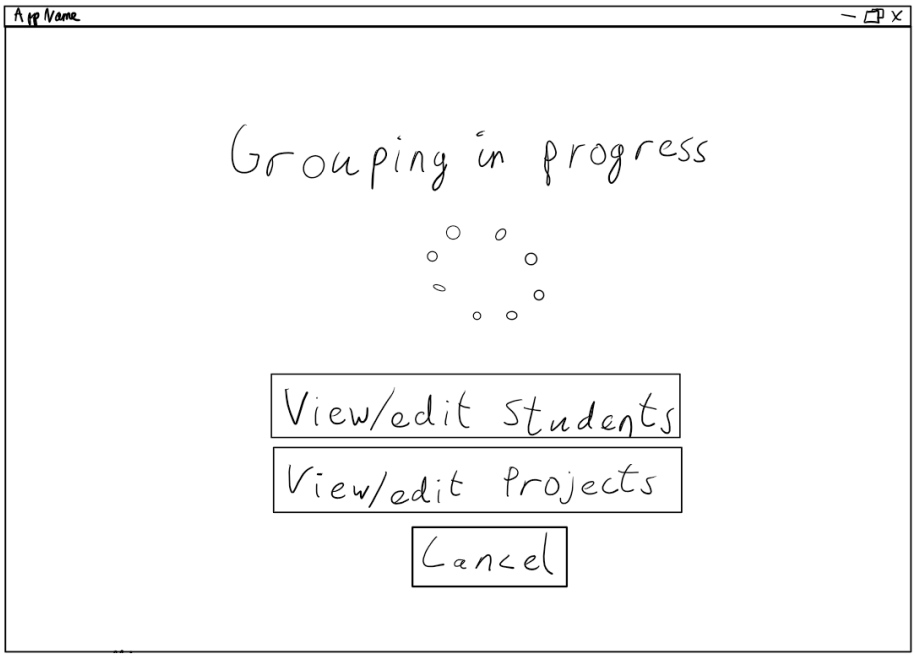


Figure 14: Sketch of the loading page layout. This page features a loading animation, and three buttons. The "View/edit Students" and "View/edit Projects" buttons lead to the Students and Projects pages, respectively [Obj. 1g]. The "Cancel" button ends the algorithm and leads to the home page. Once the algorithm has finished successfully, it moves onto the results page.

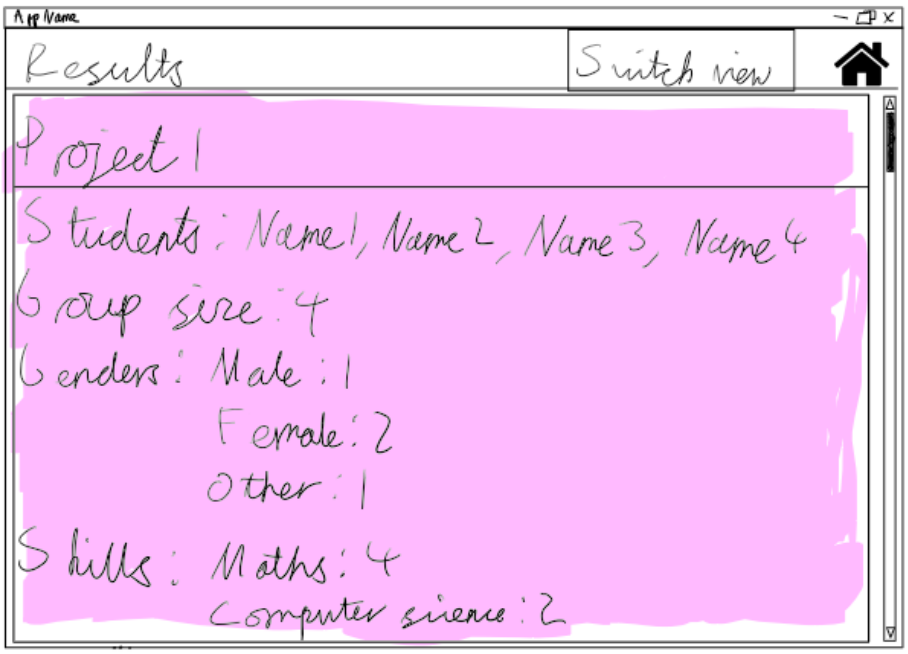


Figure 15: Sketch of the results page layout. It features a scrollable list of all the projects with the students grouped to it and summary statistics about the group, such as group size, gender ratio, and skills [Obj. 3a]. The "Switch view" leads to the register page and the home button leads to the home page.

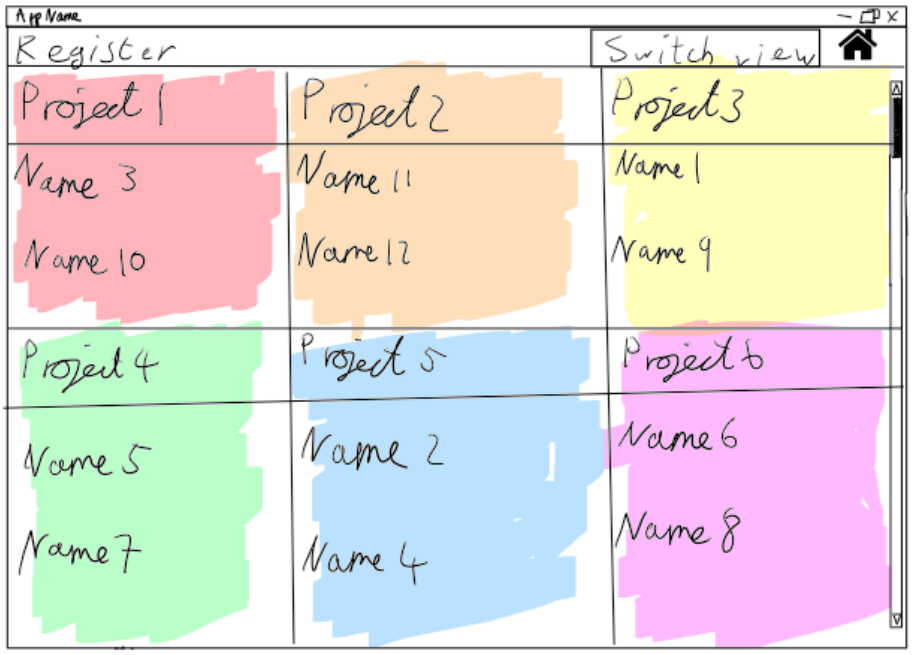


Figure 16: Sketch of the register page layout.

2.2 Databases

- Students(StudentID, Name, Gender)
- StudentSkills(StudentID, Skill)
- Projects(ProjectID, Name, MaxSize, MinSize, Optional)
- ProjectSkills(ProjectID, Skill)
- Results(ResultID, ProjectID, LocalTimeGenerated, EpochTimeGenerated)
- ResultStudentLink(ResultID, StudentID)

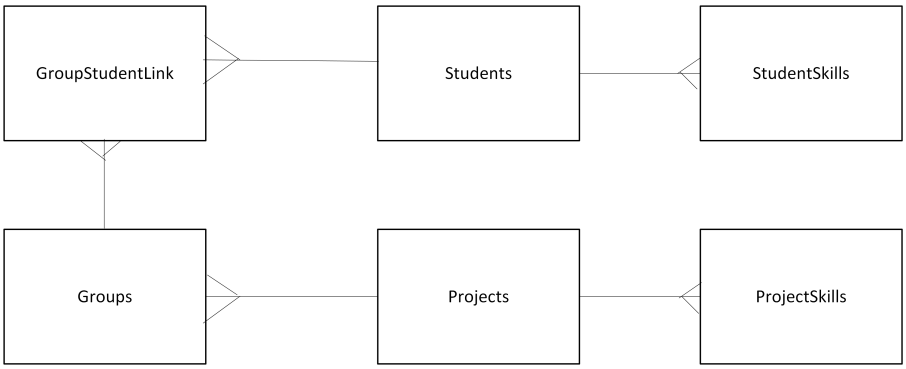


Figure 17: An entity relationship diagram showing the one-to-many relationships between the Students and Projects entities and the store of their skills to ensure all fields are atomic so the database is normalised.

2.3 Inputs and outputs

Table 1: An IPSO chart showing what data is inputted, how it is processed and stored, and how the results are outputted.

Input	Process	Storage	Output
Project details: - Name - Max group size - Min group size - Optional - Skill requirements	Validate input	Projects and ProjectSkills tables in database	Display on GUI [Fig. 12]

Student details: - Name - Gender - Skills	Validate input	Student and StudentSkills tables in database	Display on GUI [Fig. 13]
Settings: - Directory to export database to - Target fitness for genetic algorithm - Max iterations for genetic algorithm - Size of populations for genetic algorithm	Validate input	JSON file	NA
"Create Groups" button on homepage pressed	Run genetic algorithm[List. 1]	Classes for Student, Project, and Individual [Fig. 7]	Scrollable list of projects and groups [Fig. 15]

Table 2: Validations and restrictions on data entry

Data	Data Type	Validation and Restrictions
Projects		
Name	String	- Required
Max group size	Integer	- Required - Positive integers only
Min group size	Integer	- Required - Positive integers only - Must be less than or equal to 'Max group size'
Optional	Boolean	- Lack of entry is equivalent to entry of false
Skill requirements	String	- Entries separated by commas
Students		
Name	String	- Required
Gender	String	
Skills	String	- Entries separated by commas
Settings		

Directory to export database	String	- Must be a valid directory - Program must have read-/write permissions in that directory
Target fitness for genetic algorithm	Float	- Default is 0.9 - Must be between 0 and 1 inclusive
Max iterations for genetic algorithm	Integer	- Default is 50 - Must be a positive integer not including 0
Size of populations for genetic algorithm	Integer	- Default is 100 - Must be a positive integer not including 0

2.4 Algorithm

2.4.1 Pseudocode

Listing 1: Pseudocode for the function that runs when on the loading page. It ensures it is possible to find groupings with the given requirements, then runs a genetic algorithm to find a good set of groupings.

```
1  TARGET_FITNESS = integer
2  MAX_ITERATIONS = integer
3  POPULATION_SIZE = integer
4
5
6  students = set of 'student' objects
7  projects = set of 'project' objects
8
9
10 #Ensure there aren't too many or too few students to meet
       ↪ requirements
11 minStudents = 0
12 maxStudents = 0
13
14 REPEAT for all projects:
15     maxStudents += project.maxSize
16     IF (not project.optional):
17         minStudents += project.minSize
18
19 #Check for too few students
20 IF (length of students < minStudents):
21     DisplayError("Error: Only {length of students} to
           ↪ fill a minimum of {minStudents} places.")
```

```
22      HomePage()
23
24  #Check for too many students
25  ELSE IF (length of students > maxStudents): #Checks for
        ↪ too many
26      DisplayError("{length of students} is too many to fit
            ↪  in {maxStudents} places.")
27      HomePage()
28
29
30  #Generate initial grouping sets and average fitness
        ↪ scores
31  population = list
32  REPEAT for i = 0 to POPULATION_SIZE - 1:
33      groups = GenInitialGroups(projects, students)
34      avgFit = (SUM fitness of each group)/(length of
            ↪ groups)
35      APPEND groups,avgFit to population
36
37  SORT population by avgFit
38
39  #Genetic algorithm
40  REPEAT for i = 1 to MAX_ITERATIONS:
41      #Ends early when one set of groupings is good enough
42      IF (highest avgFit > TARGET_FITNESS):
43          BREAK
44
45      #Eliminate the worst half of grouping sets
46      population = first (POPULATION_SIZE FLOORDIV 2) of
            ↪ population
47      new_pop = []
48
49      #Create next population
50      REPEAT for j = 1 to 90% of POPULATION_SIZE:
51          #Crossover random groupings
52          groups = Mate(random group from population,
                ↪ random group from population)
53
54          #Ensure all students appear once
55          valid = verifyGroupings(groups)
56          IF (not valid):
57              fixGroupings(groups)
58
59          #Random mutations to prevent convergence towards
                ↪ a local best solution
60          Mutate(groups)
```

```
61
62          #Find the average fitness score for each new
                ↪ grouping
63          avgFit = (SUM fitness of each group)/(length of
                ↪ groups)
64          APPEND groups,avgFit to new_pop
65
66      #Combine the crossover with 10% elitism
67      population = new_pop + top 10% of population
68      SORT population by avgFit
69
70
71  return group with highest avgFit
```

2.4.2 Generating Initial Groups

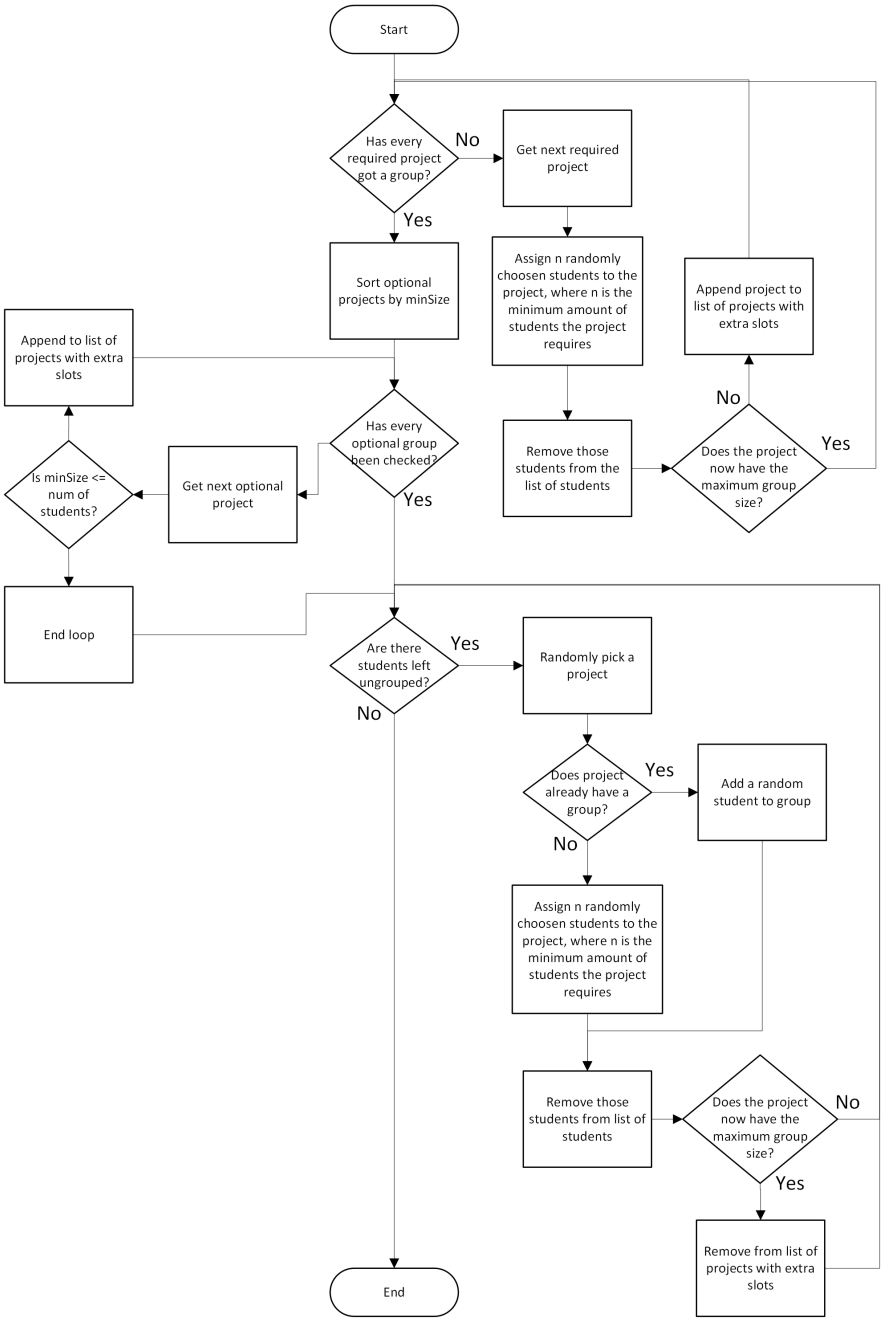


Figure 18: A flowchart for the GenInitialGroups subroutine on line 33 of the above pseudocode. It takes a list of all projects and a list of all students as parameters and returns an Individual object

2.4.3 Fitness algorithm

The fitness algorithm takes in the data about the current group and outputs a value between 0-1 that represents the suitability of the students to the project (0 being the worst possible group, 1 being absolutely perfect). This algorithm needs to allow the program to continually converge towards 1, guiding the program in the correct direction and helping it to reject bad solutions.

The factors that need to be considered by the algorithm are:

- Each student should be able to contribute to their project [Obj. 2b] – they should have at least one skill that is demanded by the project
- The demands of the project should be met by the students – the more skills are met by the students, the better the grouping
- There should be a mix of genders [Obj. 2e] – there must be at least two different genders

These factors are of different levels of importance,

2.4.4 Diagrams

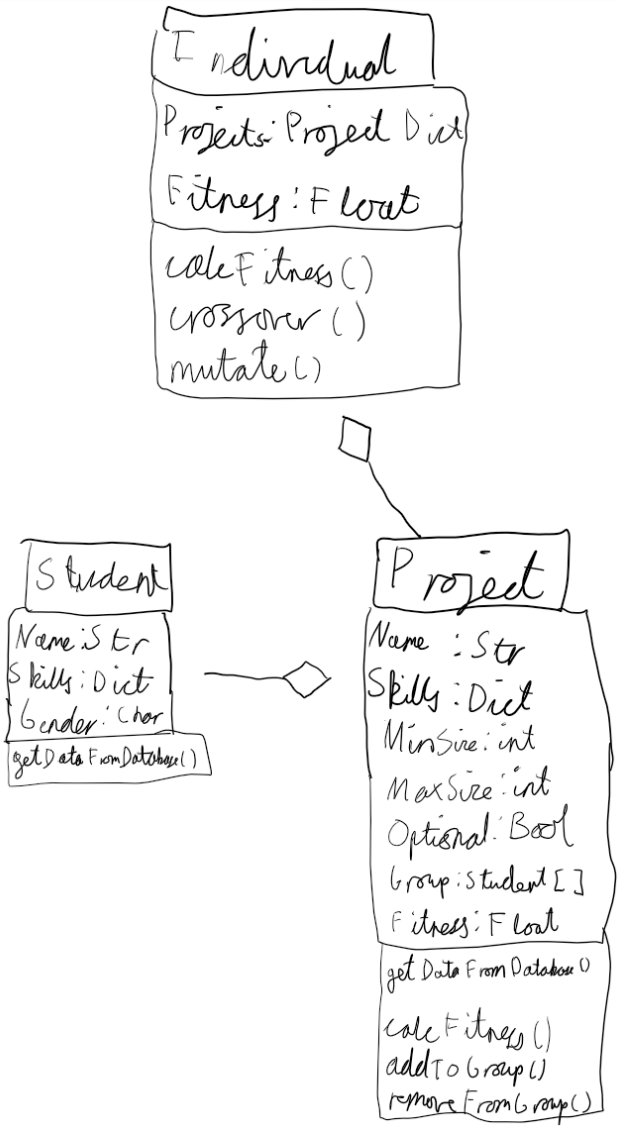


Figure 19: A class diagram showing the attributes and methods of the classes used by the genetic algorithm, and the relationships between each class.

3 Technical solution

Contents

3.1 main.py 33
3.2 algoClasses.py 53
3.3 geneticAlgo.py 55
3.4 CreateTables.sql 59

main.py is the main file that is run by the user. It contains the GUI, and calls to the functions and classes in the other files when needed. All the pages outlined above [Fig. 10] have their own classes, whose objects are created dynamically and then stored in the window class. The page classes' attributes are mostly widgets, both tkinter widgets and custom widgets classes (such as Header, ScrollableFrame, SelectionBox, and ProjectBox), or the attributes store the result of user interaction, such as the state of the checkboxes on the project page. The methods all code for the processing and outputting the result of human interaction with the GUI.

algoClasses.py contains the Individual, Student, and Project classes for use by the genetic algorithm. These follow the design of the above class diagram [Fig. 19].They are stored in a separate file for re-use across multiple files.

geneticAlgo.py contains the code for genetic algorithm that calculates the groups. it follows the design of the above pseudocode [Listing 1]

Criteria

Criteria	Location
Complex data model in database (eg several interlinked tables)	CreateTables.sql file
Complex user-defined use of OOP model	-Inheritance. E.g. projectBox inherits from selectionBox -Composition e.g. ScrollableFrame in StudentPage -Polymorphism e.g. the tkraise method in HomePage -Abstract methods e.g. get_data in selectionBox
Cross-table parameterised SQL	Initialisation of ProjectPage
Dynamic generation of objects based on complex user-defined use of OOP model	The show_frame method of the Window class
Text files	write_csv method of the SettingsPage class

Writing and reading from files	-Writing e.g. write_csv method of the SettingsPage class -Reading soon
Dictionaries	frames attribute in Window class

3.1 main.py

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
import sqlite3 as sql
import csv
import re

#=====GLOBAL CONSTANTS=====

BG_COLOUR="#FFFFFF"
HEADER_COLOUR="#FFDB32"
LARGE_FONT=("Arial",30)
DATABASE = "Code/database.db"
SQLFILEHEADER="SQLite format 3"

#=====ROOT WINDOW=====
#Constants for styling included within

class Window(tk.Tk):
    def __init__(self):
        #Set up window
        tk.Tk.__init__(self)

        self.state('zoomed')
        self.geometry(f"{self.winfo_screenwidth()-10}x{self.
    ↪ winfo_screenheight()}")
        self.title('Student Grouping Tool')
        self.config(background=BG_COLOUR)

        #=====STYLES=====

        HEADER_BUTTON_STYLE=ttk.Style()
        HEADER_BUTTON_STYLE.configure("Header.TButton",background=
    ↪ HEADER_COLOUR)

        #================
        #Dictionary that stores all loaded pages
        self.frames={}
        #Load the saved settings
        self.load_settings()
        #Load the homepage first
```

```
39          self.show_frame(HomePage)
40
41      #Method for changing currently open page
42      def show_frame(self, f):
43          #If the page is already loaded, show it
44          if f in self.frames.keys():
45              frame = self.frames[f]
46          #Otherwise, load it then show it
47          else:
48              frame = f(self)
49              self.frames[f] = frame
50              frame.place(relx=0,rely=0,relwidth=1,relheight=1)
51          frame.tkraise()
52
53      def load_settings(self):
54          #Checks for settings.json, if exists load data. If not create
    ↪ settings.json with the default values. Regardless after load
    ↪ settings page into the dict with the required settings as params.
55          frame=SettingsPage(self,0.9,50,100)
56          self.frames[SettingsPage]=frame
57          frame.place(relx=0,rely=0,relwidth=1,relheight=1)
58          return
59
60  #=====PAGES=====
61
62  class HomePage(tk.Frame):
63      def __init__(self,parent):
64          #Set up home page layout
65          tk.Frame.__init__(self, parent)
66
67          self.settingsButton = ttk.Button(self,text="Settings",command=
    ↪ lambda:parent.show_frame(SettingsPage))
68          self.settingsButton.pack()
69
70          self.label = ttk.Label(self, text="Welcome to AppName Grouping
    ↪ Tool!", font=LARGE_FONT)
71          self.label.pack()
72
73          self.projectButton = ttk.Button(self,text="Choose Projects",
    ↪ command=lambda:parent.show_frame(ProjectPage))
74          self.projectButton.pack()
75
76          self.studentButton = ttk.Button(self,text="Choose Students",
    ↪ command=lambda:parent.show_frame(StudentPage))
77          self.studentButton.pack()
78
79          self.runButton = ttk.Button(self,text="Create Groups",command=
    ↪ lambda:parent.show_frame(LoadingPage),state="disabled")
80          self.runButton.pack()
81
```

```
82          self.warning = ttk.Label(self,text="Must choose projects and
    ↪ students first")
83
84
85      #Method override that runs once everytime the page is opened
86      def tkraise(self,*args,**kwargs):
87          tk.Frame.tkraise(self)
88
89          #If at least one student and one project has been selected, allow
    ↪  the run button to be clickable and remove the warning message
90          if ProjectPage in self.master.frames and StudentPage in self.
    ↪ master.frames and self.master.frames[ProjectPage].
    ↪ check_for_selection() and self.master.frames[StudentPage].
    ↪ check_for_selection():
91              self.runButton["state"]="enabled"
92              self.warning.pack_forget()
93
94          #Otherwise, disable the run button and display the warning
    ↪ message
95          else:
96              self.runButton["state"]="disabled"
97              self.warning.pack()
98
99
100 class ProjectPage(tk.Frame):
101     def __init__(self,parent):
102         #Set up the project page layout
103         tk.Frame.__init__(self,parent)
104
105         self.header=Header(self,"Projects",HEADER_COLOUR)
106         self.header.pack(fill="x")
107
108         self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
109         self.backButton.pack(side="right")
110
111         self.importButton=ttk.Button(self,text="Import from file",command
    ↪ =self.import_projects)
112         self.importButton.pack()
113
114         self.addButton=ttk.Button(self,text="New project",command=lambda:
    ↪ self.create_entry_window("New"))
115         self.addButton.pack()
116
117         #self.separator=ttk.Separator(self,orient="horizontal")
118         #self.separator.pack(fill="x",expand=True)
119
120         self.boxesFrame=ScrollableFrame(self)
121         self.boxesFrame.canvas.pack(fill="both",expand=True)
122         self.update()
```

```
123
124         #Get and show project data from the database
125         self.projectBoxes={}
126         self.fetch_from_db(DATABASE)
127
128
129     def fetch_from_db(self,path):
130         #Fetch all project data from a database
131         query="SELECT * from Projects LEFT JOIN ProjectSkills using (
    ↪ ProjectID)"
132         with sql.connect(path) as conn:
133             cur = conn.cursor()
134             cur.execute(query)
135             projects=cur.fetchall()
136
137             for project in projects:
138                 ID=project[0]
139                 #When a project has multiple skills, the ID is repeated
    ↪ in the recieved tuple for each skill
140                 if ID in self.projectBoxes:
141                     #The next skill is appended to the existing list of
    ↪ skills for this ID
142                     data=self.projectBoxes[ID].get_data()
143
144                     skills=data["skills"]
145                     skills.append(project[5])
146                     self.projectBoxes[ID].edit_data(skills=skills)
147
148                 #If the project has not been added to the dictionary yet,
    ↪  add it
149                 else:
150                     #Unpack the data, keeping data that should not be of
    ↪ type string spearate
151                     *data,optional,skills=project
152                     #Convert to correct data type
153                     optional=bool(optional)
154                     skills=[skills]
155                     #Store data and display data on UI
156                     self.projectBoxes[ID]=projectBox(self.boxesFrame,self
    ↪ ,*data,optional,skills)
157                     self.projectBoxes[ID].pack()
158
159
160     #Method override as a work-around for a scrolling bug
161     def update(self):
162         self.master.state('normal')
163         tk.Frame.update(self)
164         self.master.state('zoomed')
165
```

```
166     #Iterates through each project, returning true as soon as it finds a
        ↪ selected project. Returns false when no project is selected.
167     def check_for_selection(self):
168         for box in self.projectBoxes:
169             if self.projectBoxes[box].check_selection():
170                 return True
171         return False
172 
173 
174     def create_entry_window(self,mode,**kwargs):
175         name=tk.StringVar()
176         maxSize=tk.StringVar()
177         minSize=tk.StringVar()
178         optional=tk.BooleanVar()
179         skills=tk.StringVar()
180         data={"name":name,"maxSize":maxSize,"minSize":minSize,"optional":
        ↪ optional,"skills":skills}
181 
182         if mode == "Edit":
183             popupWin=entryPopupWin(self,"Edit Project")
184             name.set(kwargs["name"])
185             maxSize.set(kwargs["maxSize"])
186             minSize.set(kwargs["minSize"])
187             optional.set(kwargs["optional"])
188             skills.set(", ".join(kwargs["skills"]))
189             data["ID"]=kwargs["ID"]
190         else:
191             popupWin=entryPopupWin(self,"New project")
192 
193         popupWin.add_string_entry("Name:",name)
194 
195         popupWin.add_int_entry("Maximum size allowed for this project's
        ↪ group:",maxSize)
196 
197         popupWin.add_int_entry("Minimum size allowed for this project's
        ↪ group:",minSize)
198 
199         popupWin.add_checkbox("Optional:",optional)
200 
201         popupWin.add_string_entry("Skills, each one separated by a comma:
        ↪ ",skills)
202 
203         popupWin.add_save_cancel(popupWin,mode,**data)
204 
205     def save_entry(self,window,mode,**kwargs):
206         name=kwargs["name"].get()
207         maxSize=kwargs["maxSize"].get()
208         minSize=kwargs["minSize"].get()
209         optional=kwargs["optional"].get()
210         skills=kwargs["skills"].get()
```

```
211
212         #Validation to ensure all required entries are filled
213         if name != "" and maxSize != "" and minSize != "" and skills != "
    ↪ ":
214
215             maxSize = int(maxSize)
216             minSize = int(minSize)
217
218             #Separate skills into a list and remove whitespace
219             skills=skills.split(",")
220             for i in range(len(skills)):
221                 skills[i]=skills[i].strip()
222
223             if mode == "Edit":
224                 self.edit_project(kwargs["ID"],name,maxSize,minSize,
    ↪ optional,skills)
225             else:
226                 self.new_project(name,maxSize,minSize,optional,skills)
227
228             #Destroy the popup window
229             window.destroy()
230
231
232     def import_projects(self):
233         def importtxt(path):
234             file=open(path,mode="r")
235         def importcsv(path):
236             file=open(path,newline='')
237             reader=csv.reader(file)
238
239
240         def importdb(path):
241             return
242         #Show an "Open" dialog box and return file directory
243         path = filedialog.askopenfilename() #Add excel
244         #Validation to ensure the user selected a valid file type
245         if path != None:
246             match path[-4:]:
247                 case ".txt":
248                     importtxt(path)
249                 case ".csv":
250                     importcsv(path)
251                 case _:
252                     if checkForDB(path):
253                         importdb(path)
254                     else:
255                         messagebox.showwarning(title="Import Warning",
    ↪ message="File type unrecognised!")
256
257         else:
```

```
258             messagebox.showwarning(title="Import Warning", message="No
    ↪ file selected!")
259
260
261
262     #Add the inputted data to the Project and Project Skills tables in
    ↪ the database
263     def new_project(self,name,maxSize,minSize,optional,skills):
264         mainquery="INSERT INTO Projects (Name, MaxSize, MinSize, Optional
    ↪ ) VALUES (?,?,?,?)"
265         skillsquery="INSERT INTO ProjectSkills (ProjectID, Skill) VALUES
    ↪ (?,?)"
266         with sql.connect(DATABASE) as conn:
267             cur = conn.cursor()
268
269             cur.execute(mainquery,(name,maxSize,minSize,optional))
270
271             #Fetch the autogenerated ID for this student
272             ID=cur.lastrowid
273
274             for skill in skills:
275                 cur.execute(skillsquery,(ID, skill))
276
277             conn.commit()
278
279         #Display new project on the GUI
280         self.projectBoxes[ID]=projectBox(self.boxesFrame,self,ID,name,
    ↪ maxSize,minSize,optional,skills)
281         self.projectBoxes[ID].pack()
282
283         #Work-around for a scrollbar bug
284         self.update()
285
286     def edit_project(self,ID,name,maxSize,minSize,optional,skills):
287         edited=False
288         box=self.projectBoxes[ID]
289         oldData=box.get_data()
290
291         #Only updates if the new data is different from the old data
292         with sql.connect(DATABASE) as conn:
293             cur = conn.cursor()
294
295             if name != oldData["name"]:
296                 edited=True
297
298                 query = "UPDATE Projects SET Name = ? WHERE ProjectID = ?
    ↪ "
299                 cur.execute(query,(name,ID))
300
301                 box.edit_data(name=name)
```

```
302
303             if maxSize != oldData["maxSize"]:
304                 edited=True
305
306                 query="UPDATE Projects SET MaxSize = ? WHERE ProjectID =
    ↪ ?"
307                 cur.execute(query,(maxSize,ID))
308
309                 box.edit_data(maxSize=maxSize)
310
311             if minSize != oldData["minSize"]:
312                 edited=True
313
314                 query="UPDATE Projects SET MinSize = ? WHERE ProjectID =
    ↪ ?"
315                 cur.execute(query,(minSize,ID))
316
317                 box.edit_data(minSize=minSize)
318
319             if optional != oldData["optional"]:
320                 edited=True
321
322                 query="UPDATE Projects SET Optional = ? WHERE ProjectID =
    ↪  ?"
323                 cur.execute(query,(optional,ID))
324
325                 box.edit_data(optional=optional)
326
327             if skills != oldData["skills"]:
328                 edited=True
329
330                 cur.execute("DELETE FROM ProjectSkills WHERE ProjectID =
    ↪ ?",(ID,))
331                 query = "INSERT INTO ProjectSkills (ProjectID, Skill)
    ↪ VALUES (?,?)"
332                 for skill in skills:
333                     cur.execute(query,(ID,skill))
334
335                 box.edit_data(skills=skills)
336
337             if edited:
338                 self.update()
339                 conn.commit()
340
341     def editBox(self,boxID):
342         box=self.projectBoxes[boxID]
343         data=box.get_data()
344         self.create_entry_window("Edit",ID=data["ID"],name=data["name"],
    ↪ maxSize=data["maxSize"],minSize=data["minSize"], optional=data["
    ↪ optional"], skills=data["skills"])
```

```
345
346     def deleteBox(self,boxID):
347         #Delete from database
348         query1 = "DELETE FROM Projects WHERE ProjectID=?"
349         query2 = "DELETE FROM ProjectSkills WHERE ProjectID=?"
350         with sql.connect(DATABASE) as conn:
351             cur = conn.cursor()
352             cur.execute(query1,(boxID,))
353             cur.execute(query2,(boxID,))
354             conn.commit()
355         #Delete from UI
356         self.projectBoxes[boxID].pack_forget()
357         #Delete from internal store
358         del self.projectBoxes[boxID]
359
360         self.update()
361
362
363
364 class StudentPage(tk.Frame):
365     def __init__(self,parent):
366         #Set up student page layout
367         tk.Frame.__init__(self, parent)
368
369         self.header=Header(self,"Students",HEADER_COLOUR)
370         self.header.pack(fill="x")
371
372         self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
373         self.backButton.pack(side="right")
374
375         self.importButton=ttk.Button(self,text="Import from file",command
    ↪ =self.import_students)
376         self.importButton.pack()
377
378         self.addButton=ttk.Button(self,text="New student",command=lambda:
    ↪ self.create_entry_window("New"))
379         self.addButton.pack()
380
381         self.boxesFrame=ScrollableFrame(self)
382         self.boxesFrame.canvas.pack(fill="both",expand=True)
383         self.update()
384
385         self.studentBoxes={}
386
387         #Fetch all student data from the database
388         studentquery="SELECT * from Students LEFT JOIN StudentSkills
    ↪ using (StudentID)"
389         with sql.connect(DATABASE) as conn:
390             cur = conn.cursor()
```

```
391             cur.execute(studentquery)
392             students=cur.fetchall()
393 
394             for student in students:
395                 ID=student[0]
396                 #When a student has multiple skills, the ID is repeated
    ↪ in the recieved tuple for each skill
397                 if ID in self.studentBoxes:
398                     #The next skill is appended to the existing list of
    ↪ skills for this ID
399                     data=self.studentBoxes[ID].get_data()
400                     skills=data["skills"]
401                     skills.append(student[3])
402                     self.studentBoxes[ID].edit_data(skills=skills)
403                 #If the student has not been added to the dictionary yet,
    ↪  add it
404                 else:
405                     #Unpack the data, keeping data that should not be of
    ↪ type string spearate
406                     *data,skills=student
407                     #Convert to correct data type
408                     skills=[skills]
409                     #Store data and display on UI
410                     self.studentBoxes[ID]=studentBox(self.boxesFrame,self
    ↪ ,*data,skills)
411                     self.studentBoxes[ID].pack()
412 
413     #Method override as a work-around for a scrolling bug
414     def update(self):
415         self.master.state('normal')
416         tk.Frame.update(self)
417         self.master.state('zoomed')
418 
419     def check_for_selection(self):
420         for box in self.studentBoxes:
421             if self.studentBoxes[box].check_selection():
422                 return True
423         return False
424 
425     def import_students(self):
426         #Show an "Open" dialog box and return an open file object in read
    ↪  mode
427         file = filedialog.askopenfile(mode='r',filetypes=[("Supported
    ↪ file types",".txt .csv")]) #Add excel
428         #Validation to ensure the user selected a file
429         if file == None:
430             return
431 
432         #<<<-------------------------------------------
433 
```

```
434         file.close()
435         return
436 
437     def create_entry_window(self,mode,**kwargs):
438         name=tk.StringVar()
439         gender=tk.StringVar()
440         skills=tk.StringVar()
441         data={"name":name,"gender":gender,"skills":skills}
442 
443         if mode == "Edit":
444             popupWin=entryPopupWin(self, "Edit student")
445             name.set(kwargs["name"])
446             gender.set(kwargs["gender"])
447             skills.set(", ".join(kwargs["skills"]))
448             data["ID"]=kwargs["ID"]
449         else:
450             popupWin=entryPopupWin(self,"New student")
451 
452         popupWin.add_string_entry("Name:", name)
453 
454         popupWin.add_string_entry("Gender:",gender)
455 
456         popupWin.add_string_entry("Skills, each one separated by a comma:
    ↪ ",skills)
457 
458         popupWin.add_save_cancel(popupWin,mode,**data)
459 
460     def save_entry(self,window,mode,**kwargs):
461         name=kwargs["name"].get()
462         gender=kwargs["gender"].get()
463         skills=kwargs["skills"].get()
464 
465         #Validation to ensure all required entries are filled
466         if name != "" and gender != "" and skills != "":
467 
468             #Separate skills into a list and remove whitespace
469             skills=skills.split(",")
470             for i in range(len(skills)):
471                 skills[i]=skills[i].strip()
472 
473             if mode == "Edit":
474                 self.edit_student(kwargs["ID"],name,gender,skills)
475             else:
476                 self.new_student(name,gender,skills)
477 
478             #Destroy the popup window
479             window.destroy()
480 
481 
```

```
    #Add the inputted data to the Student and StudentSkills tables in the database
    def new_student(self,name,gender,skills):
        mainquery="INSERT INTO Students (Name, Gender) VALUES (?,?)"
        skillsquery="INSERT INTO StudentSkills (StudentID, Skill) VALUES (?,?)"
        with sql.connect(DATABASE) as conn:
            cur = conn.cursor()

            cur.execute(mainquery,(name,gender))

            #Fetch the autogenerated ID for this student
            ID=cur.lastrowid

            for skill in skills:
                cur.execute(skillsquery,(ID, skill))

            conn.commit()

        #Display new student on the GUI
        self.studentBoxes[ID]=studentBox(self.boxesFrame,self,ID,name,gender,skills)
        self.studentBoxes[ID].pack()

        #Scroll bar bug work-around
        self.update()


    def edit_student(self,ID,name,gender,skills):
        edited=False

        box=self.studentBoxes[ID]
        oldData=box.get_data()

        #Only updates if the new data is different from the old data
        with sql.connect(DATABASE) as conn:
            cur = conn.cursor()

            if name != oldData["name"]:
                edited=True

                query = "UPDATE Students SET Name = ? WHERE StudentID = ? "
                cur.execute(query,(name,ID))

                box.edit_data(name=name)

            if gender != oldData["gender"]:
                edited=True

```

```
                query="UPDATE Students SET Gender = ? WHERE StudentID = ?
    ↪ "
                cur.execute(query,(gender,ID))

                box.edit_data(gender=gender)

            if skills != oldData["skills"]:
                edited=True

                cur.execute("DELETE FROM StudentSkills WHERE StudentID =
    ↪ ?",(ID,))
                query = "INSERT INTO StudentSkills (StudentID, Skill)
    ↪ VALUES (?,?)"
                for skill in skills:
                    cur.execute(query,(ID,skill))

                box.edit_data(skills=skills)

            if edited:
                self.update()
                conn.commit()

    def editBox(self,boxID):
        box=self.studentBoxes[boxID]
        data=box.get_data()
        self.create_entry_window("Edit", ID=data["ID"],name=data["name"],
    ↪ gender=data["gender"],skills=data["skills"])

    def deleteBox(self,boxID):
        #Delete from database
        query1 = "DELETE FROM Students WHERE StudentID=?"
        query2 = "DELETE FROM StudentSkills WHERE StudentID=?"
        with sql.connect(DATABASE) as conn:
            cur = conn.cursor()
            cur.execute(query1,(boxID,))
            cur.execute(query2,(boxID,))
            conn.commit()
        #Delete from UI
        self.studentBoxes[boxID].pack_forget()
        #Delete from internal store
        del self.studentBoxes[boxID]

        self.update()



class LoadingPage(tk.Frame):
    def __init__(self,parent):
        #Set up loading page layout
        tk.Frame.__init__(self, parent)
```

```
574
575         self.label = ttk.Label(self, text="Grouping in progress...")
576         self.label.pack()
577
578         self.cancelButton=ttk.Button(self,text="Cancel",command=lambda:
    ↪ parent.show_frame(HomePage))
579         self.cancelButton.pack()
580
581     #Method override that runs once everytime the page is opened
582     def tkraise(self,*args,**kwargs):
583         tk.Frame.tkraise(self,args,kwargs)
584         #result=run(vars)
585         #if result["status"]==-1:
586             #Display error
587         #else:
588             #ResultsPage()
589
590
591 class ResultsPage(tk.Frame):
592     def __init__(self,parent):
593         #Set up results page layout
594         tk.Frame.__init__(self, parent)
595
596         self.header=Header(self,"Results",HEADER_COLOUR)
597         self.header.pack(fill="x")
598
599         self.homeButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
600         self.homeButton.pack(side="right")
601
602 class RegisterPage(tk.Frame):
603     def __init__(self,parent):
604         #Set up register page layout
605         tk.Frame.__init__(self, parent)
606
607         self.header=Header(self,"Register",HEADER_COLOUR)
608         self.header.pack(fill="x")
609
610         self.homeButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
611         self.homeButton.pack(side="right")
612
613
614 class SettingsPage(tk.Frame):
615     def __init__(self,parent,fitness,iterations,popSize):
616         #Set up swttings page layout
617         tk.Frame.__init__(self,parent)
618         self.fitness=fitness
619         self.iterations=iterations
620         self.popSize=popSize
```

```
621
622         self.header=Header(self,"Settings",HEADER_COLOUR)
623         self.header.pack(fill="x")
624
625         self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=self.go_back)
626         self.backButton.pack(side="right")
627
628         self.fitnessVar=tk.DoubleVar(value=self.fitness)
629         self.iterVar=tk.IntVar(value=self.iterations)
630         self.popSizeVar=tk.IntVar(value=self.popSize)
631
632         ttk.Button(self,text="Export database", command=self.export_db).
    ↪ pack()
633
634         #Only allows user to type a decimal number between 0 and 1, no
    ↪ other characters
635         def fitnessCallback(P):
636             if re.match("^0(\\.\\d*)?$",P) or re.match("^1(\\.0*)?$",P)
    ↪ or P == "":
637                 return True
638             return False
639
640         #Only allows the user to enter positive integers
641         def posIntCallback(P):
642             return str.isdigit(P) or P==""
643
644         fitnessValidate = (self.register(fitnessCallback), '%P')
645         posIntValidate = (self.register(posIntCallback), '%P')
646
647         ttk.Label(self, text="Target fitness for genetic algorithm (Must
    ↪ be between 0-1).").pack()
648         ttk.Label(self,text="Higher target fitness means better groups,
    ↪ with 1 being perfect groupings. May not be reached if maximum
    ↪ iterations is reached").pack()
649         ttk.Spinbox(self, textvariable=self.fitnessVar,validate="all",
    ↪ validatecommand=fitnessValidate,from_=0,increment=0.05,to=1).pack
    ↪ ()
650
651         ttk.Label(self,text="Maximum iterations the genetic algorithm
    ↪ could complete (may be less if target fitness is reached).").pack
    ↪ ()
652         ttk.Spinbox(self,textvariable=self.iterVar,validate="all",
    ↪ validatecommand=posIntValidate,from_=1,increment=5,to=100).pack()
653
654         ttk.Label(self,text="Size of populations for genetic algorithm.
    ↪ Greater sizes leads to longer runtime, but better results.").pack
    ↪ ()
655         ttk.Spinbox(self,textvariable=self.popSizeVar,validate="all",
    ↪ validatecommand=posIntValidate,from_=1,increment=10,to=200).pack
```

```
    ↪ ()
656
657         ttk.Button(self, text="Save", command=self.save_settings).pack()
658
659     #Method override that runs once everytime this page is opened
660     def tkraise(self,*args,**kwargs):
661         self.fitnessVar.set(self.fitness)
662         self.iterVar.set(self.iterations)
663         self.popSizeVar.set(self.popSize)
664         tk.Frame.tkraise(self)
665
666     def get_settings(self):
667         return {"fitness":self.fitness,"iters":self.iter,"popSize":self.
    ↪ popSize}
668
669     def go_back(self): #Only show message when setting has been changed
670         choice=messagebox.askyesno("Warning","Going back will not save
    ↪ your changes. Do you still wish to proceed?",icon="warning")
671         if choice:
672             self.master.show_frame(HomePage)
673
674     def validate(self):
675         #if re.match("^0(\\.\\d+)?$",) or re.match("1(\\.0*)?$",P):
676             return True
677         #return False
678
679     #Validate and save settings input
680     def save_settings(self):
681         valid=self.validate()
682         if valid:
683             self.fitness=float(self.fitnessVar.get())
684             self.iterations=int(self.iterVar.get())
685             self.popSize=int(self.popSizeVar.get())
686             self.master.show_frame(HomePage)
687             #Update settings.json
688
689     def export_db(self): #The ui for selecting details before selecting
    ↪ directory
690         self.write_csv()
691
692     #Save database to a csv file
693     def write_csv(self):
694         filetype=[("CSV file",".csv")]
695         file = filedialog.asksaveasfile(filetypes=filetype,
    ↪ defaultextension=filetype)
696         if file is not None:
697             with sql.connect(DATABASE) as conn:
698                 cur = conn.cursor()
699                 #Fetch names of every table
```

```
                cur.execute("SELECT name FROM sqlite_schema WHERE type ='
    ↪ table' AND name NOT LIKE 'sqlite_%';")
                tables=cur.fetchall()
                #Fetch all the data from each table
                for table in tables:
                    data=cur.execute("SELECT * FROM "+str(table[0]))

                    #Write to the csv
                    writer = csv.writer(file)
                    writer.writerow("")
                    writer.writerow(table)
                    #Get attribute names
                    cols=[]
                    for col in data.description:
                        cols.append(col[0])
                    writer.writerow(cols)
                    writer.writerows(data)
        else:
            #Warn the user export was not successful
            return
        file.close()

#=====WIDGETS=====

class Header(tk.Frame):
    def __init__(self,parent,text,colour):
        tk.Frame.__init__(self,parent)
        self.config(bg=colour)
        self.label = tk.Label(self, text=text, bg=colour,font=LARGE_FONT)
        self.label.pack(side="left")

class ScrollableFrame(tk.Frame):
    def __init__(self,parent):
        self.canvas=tk.Canvas(parent,background="white")
        tk.Frame.__init__(self,self.canvas)

        self.scrollbar=tk.Scrollbar(self.canvas,orient="vertical",command
    ↪ =self.canvas.yview)

        self.canvas.configure(yscrollcommand=self.scrollbar.set)

        self.scrollbar.pack(side="right",fill="y")
        self.scrollbar.update()

        self.canvas.create_window((0,0),window=self,anchor='nw',width=
    ↪ self.winfo_screenwidth()-self.scrollbar.winfo_width())
        self.canvas.bind("<Configure>",self.scroll)
        #self.canvas.bind_all("<MouseWheel>", self._on_mousewheel)
    #def _on_mousewheel(self, event):
        #self.canvas.yview_scroll(int(-1*(event.delta/120)), "units")
```

```
747     def scroll(self,event):
748         self.canvas.configure(scrollregion=self.canvas.bbox("all"),width=
    ↪ self.winfo_width(),height=self.winfo_height())
749 
750 #Has checkbox, name, edit button, delete button.
751 class selectionBox(tk.Frame):
752     def __init__(self,parent,page,ID,name):
753         tk.Frame.__init__(self,parent,relief="solid",bd=2)
754 
755         self.ID=ID
756         self.name=name
757         self.page=page
758 
759         self.selected=tk.BooleanVar(value=False)
760         self.checkbox=ttk.Checkbutton(self,variable=self.selected,onvalue
    ↪ =True,offvalue=False)
761         self.checkbox.pack()
762 
763         self.nameLabel=ttk.Label(self,text=name)
764         self.nameLabel.pack()
765 
766         self.editButton=ttk.Button(self,text="Edit",command=self.edit)
767         self.editButton.pack()
768 
769         self.deleteButton=ttk.Button(self,text="Delete",command=self.
    ↪ delete)
770         self.deleteButton.pack()
771 
772     def check_selection(self):
773         return self.selected.get()
774 
775     def edit(self):
776         self.page.editBox(self.ID)
777 
778     def delete(self):
779         delChoice=messagebox.askyesno(title="Delete entry", message=f"Are
    ↪  you sure you want to delete {self.name}?")
780         if delChoice:
781             self.selected.set(False)
782             self.page.deleteBox(self.ID)
783 
784     def pack(self):
785         tk.Frame.pack(self,fill="x",padx=10,pady=5)
786 
787     #-----Abstract methods-----
788     def get_data(self):
789         return
790 
791     def edit_data(self):
792         return
```

```
793
794
795 class projectBox(selectionBox):
796     def __init__(self,parent,page,ID,name,maxSize,minSize,optional,skills
    ↪ ):
797         selectionBox.__init__(self,parent,page,ID,name)
798         self.maxSize=maxSize
799         self.minSize=minSize
800         self.optional=optional
801         self.skills=skills
802
803         self.maxSizeLabel=ttk.Label(self,text=f"Max group size: {maxSize}
    ↪ ")
804         self.maxSizeLabel.pack()
805
806         self.minSizeLabel=ttk.Label(self,text=f"Min group size: {minSize}
    ↪ ")
807         self.minSizeLabel.pack()
808
809         self.optionalLabel=ttk.Label(self,text=f"Optional: {optional}")
810         self.optionalLabel.pack()
811
812         self.skillsLabel=ttk.Label(self,text=f"Skills: {", ".join(skills)
    ↪ }")
813         self.skillsLabel.pack()
814
815     def get_data(self):
816         return {"ID":self.ID,"name":self.name,"maxSize":self.maxSize,"
    ↪ minSize":self.minSize,"optional":self.optional,"skills":self.
    ↪ skills}
817
818     def edit_data(self,**kwargs):
819         if "name" in kwargs:
820             self.name=kwargs["name"]
821             self.nameLabel.config(text=self.name)
822         if "maxSize" in kwargs:
823             self.maxSize=kwargs["maxSize"]
824             self.maxSizeLabel.config(text=f"Max group size: {self.maxSize
    ↪ }")
825         if "minSize" in kwargs:
826             self.minSize=kwargs["minSize"]
827             self.minSizeLabel.config(text=f"Min group size: {self.minSize
    ↪ }")
828         if "optional" in kwargs:
829             self.optional=kwargs["optional"]
830             self.optionalLabel.config(text=f"Optional: {self.optional}")
831         if "skills" in kwargs:
832             self.skills=kwargs["skills"]
833             self.skillsLabel.config(text=f"Skills: {", ".join(self.skills
    ↪ )}")
```

```
834
835 class studentBox(selectionBox):
836     def __init__(self, parent, page, ID, name, gender, skills):
837         selectionBox.__init__(self,parent,page,ID,name)
838         self.gender=gender
839         self.skills=skills
840
841         self.genderLabel = ttk.Label(self,text=f"Gender: {gender}")
842         self.genderLabel.pack()
843
844         self.skillsLabel = ttk.Label(self, text= f"Skills: {", ".join(
    ↪ skills)}")
845         self.skillsLabel.pack()
846
847     def get_data(self):
848         return {"ID":self.ID,"name":self.name,"gender":self.gender,"
    ↪ skills":self.skills}
849
850     def edit_data(self,**kwargs):
851         if "name" in kwargs:
852             self.name=kwargs["name"]
853             self.nameLabel.config(text=self.name)
854         if "gender" in kwargs:
855             self.gender=kwargs["gender"]
856             self.genderLabel.config(text=f"Gender: {self.gender}")
857         if "skills" in kwargs:
858             self.skills=kwargs["skills"]
859             self.skillsLabel.config(text=f"Skills: {", ".join(self.skills
    ↪ )}")
860
861
862 class entryPopupWin(tk.Toplevel):
863     def __init__(self,parent,title):
864         tk.Toplevel.__init__(self,parent)
865         self.title(title)
866
867     def add_string_entry(self,text,variable):
868         ttk.Label(self, text=text).pack()
869         ttk.Entry(self, textvariable=variable).pack()
870
871     def add_int_entry(self,text,variable,minimum=0,maximum=100,increment=
    ↪ 1):
872
873         #Prevents user from typing non numbers into number-only entry
    ↪ boxes
874         def intCallback(P):
875             return str.isdigit(P) or P == ""
876
877         intValidate = (self.register(intCallback), '%P')
878
```

```
879         ttk.Label(self, text=text).pack()
880         ttk.Spinbox(self, textvariable=variable,validate="all",
    ↪ validatecommand=intValidate,from_=minimum,increment=increment,to=
    ↪ maximum).pack()
881 
882     def add_checkbox(self,text,variable):
883         ttk.Label(self, text=text).pack()
884         ttk.Checkbutton(self, variable=variable, onvalue=True, offvalue=
    ↪ False).pack()
885 
886     def add_save_cancel(self,window,mode,**kwargs):
887         ttk.Button(self, text="Save", command=lambda:self.master.
    ↪ save_entry(window,mode,**kwargs)).pack()
888         ttk.Button(self, text="Cancel", command=self.destroy).pack()
889 
890 
891 #=====SUBROUTINES=====
892 def checkForDB(path):
893     #Database files could have any extension, but they all start with the
    ↪  same first 15 characters.
894     file=open(path,"rb")
895     line=file.read(15)
896     file.close()
897     #Convert binary to string
898     line="".join(chr(x) for x in line)
899     if line == "SQLite format 3":
900         return True
901     return False
902 
903 
904 
905 #=====MAIN=====
906 
907 app=Window()
908 app.mainloop()
```

3.2 algoClasses.py

```
1 class Project:
2     def __init__(self,name,skills,maxSize,minSize,optional,group=[]):
3         self.name=name
4         self.skills=skills
5         self.maxSize=maxSize
6         self.minSize=minSize
7         self.optional=optional
8 
9         self.group=group
```

```
10          self.fitness=self.calcFitness()
11
12      def calcFitness(self):
13          return
14
15      def addStudent(self,students):
16          if type(students)==list:
17              self.group+=students
18
19          elif type(students)==Student:
20              self.group.append(students)
21
22          else:
23              raise TypeError(f"Expected data with type list or Student,
    ↪ got data of type {type(students)}")
24
25          self.fitness=self.calcFitness()
26
27
28  class Student:
29      def __init__(self,name,skills,gender):
30          self.name=name
31          self.skills=skills
32          self.gender=gender
33
34
35  class Individual:
36      def __init__(self,projects=[]):
37          self.projects=projects
38          self.fitness=self.calcFitness()
39
40      def calcFitness(self):
41          return
42
43      def addProject(self,projects):
44          if type(projects)==list:
45              self.projects+=projects
46
47          elif type(projects)==Project:
48              self.projects.append(projects)
49
50          else:
51              raise TypeError(f"Expected data with type list or Project,
    ↪ got data of type {type(projects)}")
52
53          self.fitness=self.calcFitness()
```

3.3 geneticAlgo.py

```
from algoClasses import Project,Student,Individual
import sqlite3 as sql
import random

def get_students(database,IDs):
     students=[]

     with sql.connect(database) as conn:
          cur = conn.cursor()

          for ID in IDs:
               #Get each students' skills
               cur.execute("SELECT Skill FROM StudentSkills WHERE
    ↪ StudentID = ?",(ID,))
               skills=cur.fetchall()[0]

               #Get each students' name and gender
               cur.execute("SELECT Name, Gender from Students WHERE
    ↪ StudentID = ?",(ID,))
               entry=cur.fetchall()[0]

               #Create a Student object for each student with their name,
    ↪  skills, and gender
               students.append(Student(entry[0],skills,entry[1]))

     return students


def get_projects(database,IDs):
     projects={"optional":[],"notOptional":[]}

     with sql.connect(database) as conn:
          cur = conn.cursor()

          for ID in IDs:
               #Get each projects' skill requirements
               cur.execute("SELECT Skill FROM ProjectSkills WHERE
    ↪ ProjectID = ?",(ID,))
               skills=cur.fetchall()[0]

               #Get each projects' data other than skills
               cur.execute("SELECT Name, MaxSize, MinSize, Optional from
    ↪ Projects WHERE ProjectID = ?",(ID,))
               entry=cur.fetchall()[0]

               #Create a Project object for each project with its name,
    ↪ skills, maxSize, minSize, and whether or not it is optional
```

```
42                if bool(entry[3]):
43                     projects["optional"].append(Project(entry[0],skills,
    ↪ entry[1],entry[2],entry[3]))
44                else:
45                     projects["notOptional"].append(Project(entry[0],
    ↪ skills,entry[1],entry[2],entry[3]))
46
47
48      return projects
49
50 #find optional projects whose minSize is not too large to house the
    ↪ remaining ungrouped students
51 def get_optionals(optionals,extraSlots):
52      optionals.sort(key=lambda x:x.minSize, reverse=True)
53      for i in range(len(optionals)):
54           if optionals[i].minSize>extraSlots:
55                optionals.pop(i)
56           else:
57                break
58      return optionals
59
60
61 def GenInitialGroups(projects,students):
62      individual=Individual()
63      leftover=[] #Stores projects that have groups, but do not have the
    ↪ maximum number of students assigned yet
64
65      #Fill every required project to minSize
66      required=projects["notOptional"]
67      for project in required:
68           random.shuffle(students)
69           project.addStudent(students[:project.minSize])
70           individual.addProject(project)
71
72           #Remove grouped students
73           students=students[project.minSize:]
74
75           #Keep projects with room to spare so that they may take on more
    ↪  students later
76           if project.minSize!=project.maxSize:
77                leftover.append(project)
78
79      #Ensure every student is grouped
80      optionals=projects["Optional"] #Stores projects that do not yet have
    ↪  a group assigned
81
82      while len(students)>0:
83           #Only allow students to be grouped to a project with a minSize
    ↪ that allows the remaining students to be housed
84           optionals=get_optionals(optionals,len(students))
```

```
85
86          #Add random students to a random project
87          project=random.choice(leftover+optionals)
88          random.shuffle(students)
89
90          #If the project has no group currently, add minSize number of
    ↪ students
91          if project in optionals:
92               optionals.remove(project)
93               project.addStudent(students[:project.minSize])
94               individual.addProject(project)
95
96               students=students[project.minSize:]
97
98               #If project is not full
99               if project.minSize!=project.maxSize:
100                    leftover.append(project)
101
102         #If project already has a group, add one student to it
103         else:
104              project.addStudent(students[0])
105              students.pop(0)
106
107              #Remove from list if full
108              if len(project.group)==project.maxSize:
109                   leftover.remove(project)
110
111     return individual
112
113
114
115
116 def verifyGroupings(individual):
117     return True
118
119 def fixGroupings(individual):
120     return individual
121
122
123 def run(database, studentIDs, projectIDs, TARGET_FITNESS, MAX_ITERATIONS,
    ↪  POPULATION_SIZE):
124     #List of students
125     students=get_students(database,studentIDs)
126     #Two lists of projects in dictionary, separated by whether or not
    ↪ the project is optional
127     projects=get_projects(database,projectIDs)
128
129     #Ensure there aren't too many or too few students to meet
    ↪ requirements#
130     numStudents=len(students)
```

```
    minStudents=0
    maxStudents=0

    #Find max and min amount of students required
    for project in projects["notOptional"]:
        maxStudents += project.maxSize
        minStudents += project.minSize
    for project in projects["optional"]:
        maxStudents += project.maxSize
    #Return error for too few
    if numStudents<minStudents:
        return {"status":-1,"error":f"Error: Only {len(students)} to
    ↪ fill a minimum of {minStudents} places."}
    #Return error for too many
    elif numStudents>maxStudents:
        return {"status":-1,"error":f"Error: {len(students)} is too
    ↪ many to fit in {maxStudents} places."}

    #Generate initial groupings and average fitness
    population = []
    for i in range(POPULATION_SIZE):
        individual = GenInitialGroups(projects, students)
        population.append(individual)

    population.sort(key=lambda x:x.fitness,reverse=True)

    #Genetic algorithm
    for i in range(MAX_ITERATIONS):
        #Ends early when an individual has reached the target fitness
        if population[0].fitness>=TARGET_FITNESS:
            return {"status":0,"results":population[0].projects}

        #Eliminate individuals in the bottom half in terms of fitness
        population = population[:POPULATION_SIZE//2]
        new_pop = []

        #Create next population
        for j in range(int(0.9*POPULATION_SIZE)):
            #Crossover random individuals
            parents=random.sample(population, 2)
            individual=parents[0].crossover(parents[1])

            #Ensure all students appear once
            valid = verifyGroupings(individual)
            if not valid:
                individual=fixGroupings(individual)

            #Random mutations to prevent convergence towards a local
    ↪ best solution
            individual=individual.mutate()
```

```
178
179                 new_pop.append(individual)
180
181             #Create the next population with the crossover population and
    ↪ the best 10% of the last population
182             population=population[:POPULATION_SIZE*0.1]
183             population+=new_pop
184
185
186
187
188 if __name__=="__main__":
189      run("Code/database.db",[1,3,4,5,6],[4,5,6],0.9,50,10)
```

3.4 CreateTables.sql

```
1  -- SQLite
2  PRAGMA foreign_keys = ON;
3
4  CREATE TABLE IF NOT EXISTS Students (
5      StudentID INTEGER PRIMARY KEY,
6      Name TEXT,
7      Gender TEXT
8  );
9
10 CREATE TABLE IF NOT EXISTS StudentSkills (
11     StudentID INTEGER NOT NULL,
12     Skill TEXT NOT NULL,
13   PRIMARY KEY (StudentID, Skill),
14   FOREIGN KEY (StudentID) REFERENCES Students (StudentID)
15         ON DELETE CASCADE
16         ON UPDATE CASCADE
17 );
18
19 CREATE TABLE IF NOT EXISTS Projects (
20     ProjectID INTEGER PRIMARY KEY,
21     Name TEXT,
22     MaxSize INTEGER,
23     MinSize INTEGER,
24     Optional INTEGER
25 );
26
27 CREATE TABLE IF NOT EXISTS ProjectSkills (
28     ProjectID INTEGER NOT NULL,
29     Skill TEXT NOT NULL,
30   PRIMARY KEY (ProjectID, Skill),
31   FOREIGN KEY (ProjectID) REFERENCES Projects (ProjectID)
```

```
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS Results (
    ResultID INTEGER PRIMARY KEY,
    ProjectID INTEGER NOT NULL,
    LocalTimeGenerated TEXT,
    EpochTimeGenerated INTEGER,
    FOREIGN KEY (ProjectID) REFERENCES Projects (ProjectID)
        ON DELETE CASCADE
        ON UPDATE CASCADE

);

CREATE TABLE IF NOT EXISTS ResultStudentLink (
    ResultID INTEGER NOT NULL,
    StudentID INTEGER NOT NULL,
  PRIMARY KEY (ResultID, StudentID),
    FOREIGN KEY (ResultID) REFERENCES Results (ResultID)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
  FOREIGN KEY (StudentID) REFERENCES Students (StudentID)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

4 Testing

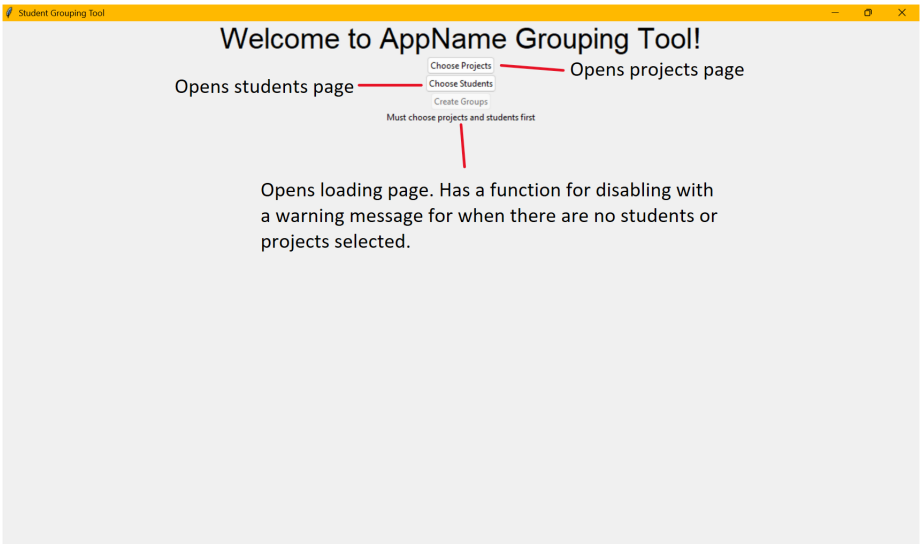


Figure 20: The home page

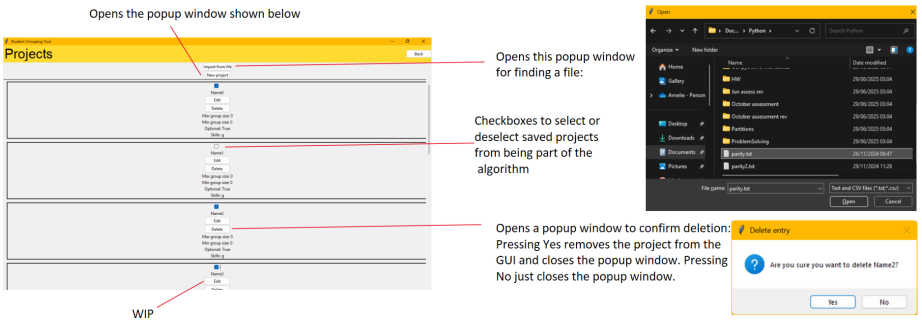


Figure 21: The project page

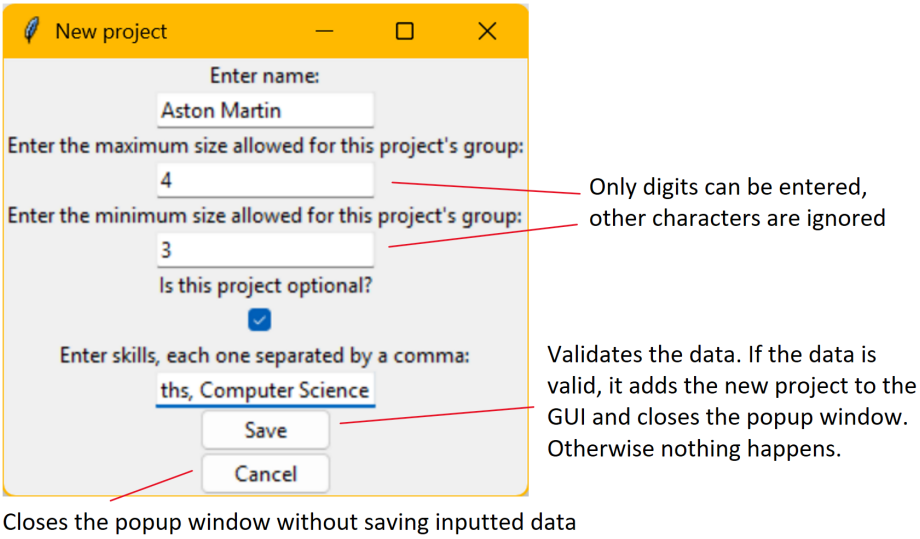


Figure 22: The popup window for entering data about a new project

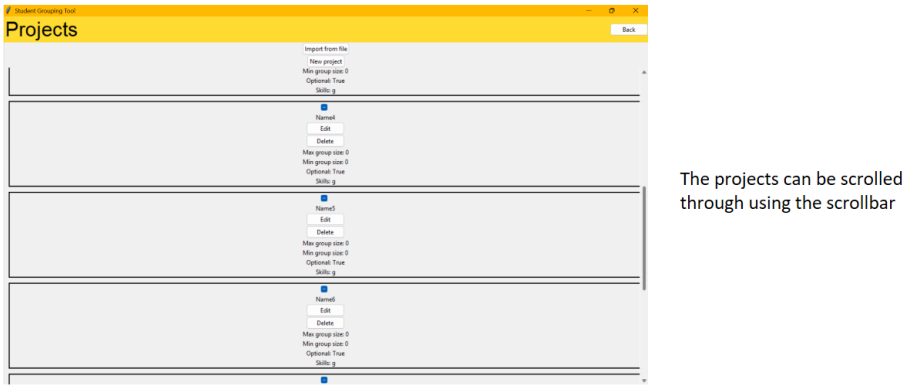


Figure 23: The project page, scrolled down

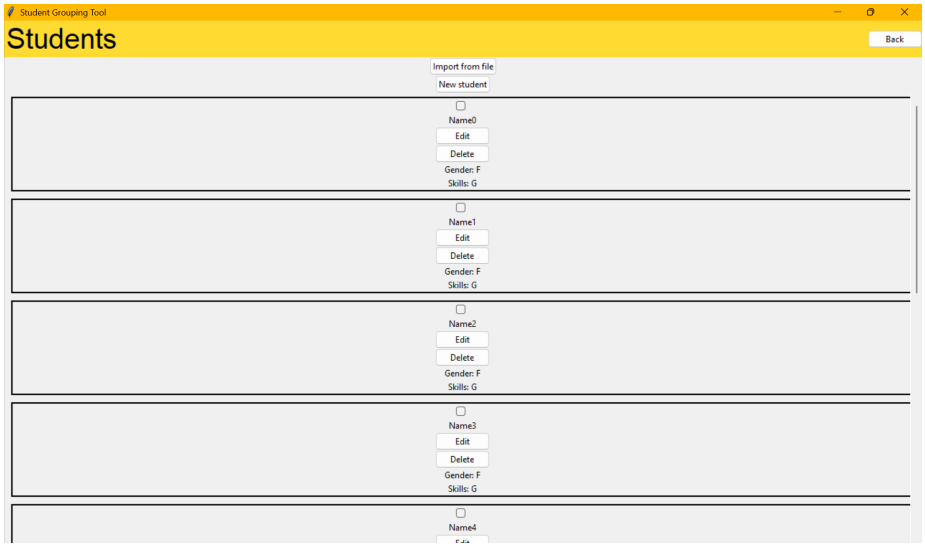


Figure 24: The student page. Functions identically to the projects page

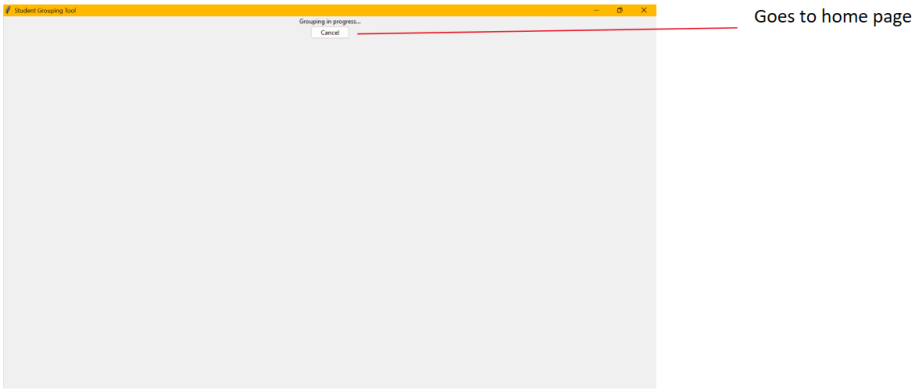


Figure 25: The loading page

4.1 Video

`https://youtu.be/5m5Lk-DWQd8`

4.2 Testing plan

ID	Obj.	Description	Test Data	Expected Results	Time stamp
1	1a,1ei	Check that the student's name must only contain whitelisted characters when entered manually			
2	1a,1e	Check that all student fields are required			
3	1a,1e,4a	Check that the entered student data is saved when all fields are completed correctly			
4	1h,4a	Check that student data can be edited			
5	1h,4a	Check that student data can be deleted			
6	1c	Check that student data is saved from previous uses of the program			
7	1bi,1d	Check that student data can be imported through csv files when all fields are correct			
8	1bi,1ei	Check that student name must only contain whitelisted characters when importing a csv file			
9	1bi,1e	Check that all student fields are required when importing a csv file			
10	1a,1fi	Check that the project's name must only contain whitelisted characters when entered manually			

11	1a,1fii	Check that the project's maximum group size must be greater than minimum group size when entered manually			
12	1a,1fiii	Check that the project's minimum group size must be greater than 0 when entered manually			
13	1a,1f	Check that all project fields are required			
14	1a,1f,4a	Check that the entered project data is saved when all fields are completed correctly			
15	1h,4a	Check that project data can be edited			
16	1h,4a	Check that project data can be deleted			
17	1c	Check that project data is saved from previous uses of the program			
18	1bi,1d	Check that project data can be imported through csv files when all fields are correct			
19	1bi,1fi	Check that project name must only contain whitelisted characters when importing a csv file			
20	1bi,1fii	Check that the project's maximum group size must be greater than minimum group size when importing a csv file			
21	1bi,1fiii	Check that the project's minimum group size must be greater than 0 when importing a csv file			
22	1bi,1f	Check that all project fields are required when importing a csv file			

23	1gi	Check that setting's target fitness must be between 0 and 1			
24	1gii	Check that setting's max iterations must be a positive integer, not including 0			
25	1giii	Check that setting's size of populations must be a positive integer, not including 0			
26	4bi	Check that database can be exported as a csv file			
27	4c	Check that settings data is stored as a JSON			
28	1i	Check that at least one student, one project, target fitness, max iterations, and size of populations must all be entered before the algorithm is run			

5 Evaluation

5.1 Self reflection

5.2 Third-party feedback